UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Viviane Duarte Bonfim
Fabiane Barreto Vavassori Benitti

RELATÓRIO TÉCNICO
ONTORED - ONTOLOGIA DE DOMÍNIO VOLTADA ÀS DÍVIDAS DE REQUISITOS

FLORIANÓPOLIS - SC
2026

LISTA DE ILUSTRAÇÕES

FIGURA 1 - Estrutura das Classes da OntoReD 10
FIGURA 2 - Estrutura dos elementos da prática 11
FIGURA 3 - Elementos da prática Revisão de Requisitos 13
FIGURA 4 - Sequência das atividades da prática 17
FIGURA 5 - Elementos da prática - Manter e descrever com clareza o documento de especificação de requisitos 19
FIGURA 6 - Definição de requisitos de usuário 25
FIGURA 7 - Classificação de requisitos de qualidade (não funcionais) 28
FIGURA 8 - Exemplos de requisitos de qualidade 28
FIGURA 9 - Exemplo de descrição de cenário 32
FIGURA 10 - Exemplo de diagrama de caso de uso 33
FIGURA 11 - Exemplo 1 de diagrama de atividade 34
FIGURA 12 - Exemplo 2 de diagrama de atividade 34
FIGURA 13 - Fluxo de atividades da prática “Manter e descrever com clareza o documento de especificação de requisitos” 35
FIGURA 14 - Estrutura da Prática “Manter Rastreabilidade dos requisitos” 36
FIGURA 15 - Categorias das atividades do gerenciamento de requisitos 37
FIGURA 16 - Exemplo 1 de pré e pós-rastreabilidade dos requisitos 41
FIGURA 17 - Exemplo 2 de pré e pós-rastreabilidade dos requisitos 41
FIGURA 18 - Exemplo de rastreabilidade horizontal entre versões do mesmo artefato 42
FIGURA 19 - Exemplo de rastreabilidade vertical entre requisitos e artefatos produzidos em diferentes fases do projeto 42
FIGURA 20 - Exemplo 1 de matriz de rastreabilidade entre requisitos 44
FIGURA 21 - Exemplo 2 de matriz de rastreabilidade entre requisitos 44
FIGURA 22 - Exemplo 3 de matriz de rastreabilidade de requisitos elaborada em planilha eletrônica 45
FIGURA 23 - Exemplo de Grafo de rastreabilidade de requisitos 47
FIGURA 24 - Sequência das atividades da prática 49
FIGURA 25 - Estrutura da prática “Treinamento de padrões de elicitação de requisitos” 51
FIGURA 26 - Protocolo do garçom para filtrar falhas de comunicação na elicitação de requisitos 59
FIGURA 27 - Exemplo de user stories 63
FIGURA 28 - Questões para identificação dos stakeholders 68
FIGURA 29 - Sequência das atividades da prática 69
FIGURA 30 - Hierarquia das classes da OntoReD 71
FIGURA 31 - *Objects Properties* 72

FIGURA 32 - Classes da OntoReD implementadas no Protegé e exemplos da relação entre elas por meio das *Objects Properties* 73
FIGURA 33 - *Data property* das instâncias 74
FIGURA 34 - Exemplos de Instâncias da classe "CausaReD" da OntoReD 76
FIGURA 35 - Resultado Consulta 1: Consequências da dívida de requisitos mitigadas pela prática "Treinamento de padrões de elicitação de requisitos" 77
FIGURA 36 - Resultado Q1. Quais causas da dívida ocasiona(m) determinada Consequência? 79
FIGURA 37 - Resultado Q2. Quais consequências são causadas por determinada causa? 79
FIGURA 38 - Resultado Q2. Quais consequências são causadas por determinada causa? 80
FIGURA 39 - Resultado Q3. Quais são as práticas que podem mitigar determinada(s) causa(s) da dívida de requisitos? 80
FIGURA 40 - Resultado Q4. Quais são as práticas que podem mitigar determinada(s) causa(s) da dívida de requisitos? 80
FIGURA 41 - Resultado Q5. Quais práticas estão associadas a determinada área da gestão da dívida? 81
FIGURA 42 - Resultado Q6. Qual área da gestão da dívida está associada a determinada prática que pode mitigar determinada causa? 81

LISTA DE QUADROS

QUADRO 1 - Especificação da OntoReD.....9
QUADRO 2 - Elementos da Prática conforme Linguagem Essence (OMG, 2018) ..11
QUADRO 3 - Características das *Objects Properties*.....72
QUADRO 4 - *Objects Properties Domain e Range*.....73
QUADRO 5 - Axioma voltado às práticas que podem mitigar as causas das dívidas de requisitos 75
QUADRO 6 - Exemplos de Instâncias da OntoReD.....75
QUADRO 7 - Exemplo de consulta da OntoReD em DL Query.....77
QUADRO 8 - Respostas às questões previstas na especificação da OntoReD, conforme Quadro 1, Seção 2.3 78
QUADRO 9 - Exemplos de Instâncias da OntoReD.....82

SUMÁRIO

LISTA DE ILUSTRAÇÕES ..... **II**

LISTA DE QUADROS ..... **IV**

1 INTRODUÇÃO **6**

2 ONTORED - PROPOSTA DE UMA ONTOLOGIA DE DOMÍNIO VOLTADA ÀS DÍVIDAS DE REQUISITOS **7**

2.1 Apresentação da OntoReD 7

2.2 Construção da OntoReD baseada na *Methontology* 8

2.3 Especificação 9

2.4 Conceitualização 10

2.5 Conceitualização – Detalhamento dos elementos das práticas específicas 12

2.5.1 Elaborar diagrama de caso de uso e Requisitos bem definidos registrados no backlog *12*

2.5.2 Prática: Revisão de requisitos *13*

2.5.3 Prática: Manter e descrever com clareza o documento de especificação de requisitos 18

2.5.4 Prática: Manter a rastreabilidade dos requisitos *35*

2.5.5 Prática: Treinamento de padrões de elicitação de requisitos *51*

2.6 Formalização e Implementação 70

2.6.1 Construção das classes *71*

2.6.2 Definição dos Relacionamentos da OntoReD – Object Properties *71*

2.6.3 Definição dos atributos de instâncias da OntoReD – Data Properties *74*

2.6.4 Axiomas *74*

2.6.5 Definição das instâncias das Classes da OntoReD *75*

2.7 Avaliação da OntoReD 76

2.8 Documentação da OntoReD 81

2.9 Glossário da OntoReD 82

REFERÊNCIAS **85**

1 INTRODUÇÃO

Este relatório técnico consiste em fornecer a documentação da Ontologia denominada OntoReD, proposta por Bonfim e Benitti (2025), destinada a organizar e divulgar o conhecimento sobre dívidas de requisitos, considerando a gestão da dívida, as causas das dívidas de requisitos e suas consequências, bem como a recomendação de práticas para mitigá-las.

Por meio da OntoReD é possível: I) determinar um vocabulário comum especializado voltado à dívida de requisitos, caracterizando o domínio desta ontologia; II) sistematizar o conhecimento, pois requer um conjunto de conceitos que permitem descrever elementos fundamentais da área base; III) padronizar, pois promove a relação do vocabulário comum e a sistematização do conhecimento para representar os conceitos que seguem uma padronização facilitando a sua compreensão (Mizoguchi, 2001).

A OntoReD proposta foi motivada para servir de fonte de conhecimento e aproximar o conhecimento, voltado às dívidas de requisitos, das organizações ágeis. Por meio do conhecimento da Ontologia as organizações ágeis poderão consultar as causas que levam às consequências das dívidas e compreender melhor o seu cenário de requisitos, bem como as práticas que poderão mitigá-las. A recomendação das práticas fornecerá o seu detalhamento e orientações sobre elas.

Espera-se que a Ontologia possa potencializar a qualidade dos requisitos elicitados e especificados em organizações ágeis, evitando o retrabalho, reduzindo custo adicional e, consequentemente, melhorando a qualidade dos produtos de software desenvolvidos.

2 ONTORED - PROPOSTA DE UMA ONTOLOGIA DE DOMÍNIO VOLTADA ÀS DÍVIDAS DE REQUISITOS

A OntoReD é o primeiro artefato da contribuição da tese desenvolvida por Bonfim; Benitti (2026) e visa organizar e estruturar os fundamentos teóricos voltados à dívida de requisitos, considerando conceitos (classes), organização hierárquica entre eles, suas relações e axiomas para formalizar as definições e relações, assim como objetos e seus atributos, sintetizando o modelo que representa as informações obtidas, formando a base de conhecimento da área envolvida (Fernández-Pérez; Goméz - López; Corcho, 2004).

A construção da OntoReD, reúne um vocabulário comum sobre dívidas de requisitos sintetizando o conhecimento adquirido através da *Grounded Theory* (GT) realizado por Bonfim; Benitti (2026), bem como as evidências obtidas por meio dos estudos secundário e terciário desenvolvidos pelas autoras citadas, que revela um grande volume de dados, que poderá ser compartilhado e complementado por outros pesquisadores. Dessa forma, novas informações poderão ser agregadas à OntoReD, contribuindo para a mitigação das dívidas de requisitos, a evolução do processo de requisitos e melhoria da qualidade dos requisitos especificados.

2.1 Apresentação da OntoReD

A OntoReD é uma ontologia de domínio que consiste em representar na sua base de informações, as evidências sobre as causas que geram dívidas de requisitos, as consequências ocasionadas pelas causas, práticas que podem mitigá-las, bem como aspectos voltados à gestão da dívida.

A partir da definição do vocabulário é possível manter um conjunto de elementos designados como classes ou conceitos, relações, instâncias e axiomas

(Fernández-López; Gómez-Pérez; Corcho 2004), os quais fazem parte dos elementos que embasam a construção da OntoReD e são conceituados a seguir:

• Classes ou conceitos: As classes representam os conceitos de um domínio, podem conter outras classes, seguindo uma taxonomia e possuem um conjunto de instâncias, suas relações, atributos. “CausaReD” e “ConsequenciaReD” são exemplos de classes.

• Relações: Determinam as propriedades entre as classes e, consequentemente, entre suas instâncias. As relações representam a semântica de uma ontologia. Como exemplos, apresenta-se “ocasiona” e “mitigadoPor”.

• Instâncias: Caracterizam os objetos de uma classe e a base de conhecimento de uma ontologia. “InconsistenciaRequisitos” é um exemplo de uma instância da classe exemplificada como “ConsequenciaReD”.

• Axiomas: Representam as regras que permitem associar os relacionamentos entre as instâncias.

O desenvolvimento da OntoReD é amparado pela metodologia *Methontology*, considerando os elementos fundamentais descritos anteriormente.

2.2 Construção da OntoReD baseada na *Methontology*

A *Methontology* é uma metodologia para construção de ontologias proposta por Fernández-López, Gómez-Pérez e Juristo (1997) em nível de conhecimento. Essa metodologia é embasada nas principais atividades voltadas ao processo de desenvolvimento de software e em metodologias da engenharia do conhecimento (Fernández-López; Gómez-Pérez; Corcho, 2004).

Optou-se pela *Methontology* por fornecer orientação sobre o processo de construção da ontologia (Iqbal et al., 2013) e consiste nas seguintes atividades: Especificação, conceitualização, formalização, implementação e manutenção; por ser referência no desenvolvimento de ontologias voltadas à área da engenharia de software; e, também, porque várias ontologias propostas na área de Computação se apoiam na *Methontology* para sua construção (Vale-Pereira; Fernandéz 2019; Alsanad; Chikh;

Mirza, 2019). Complementarmente ao processo sugerido pela *Methontology*, a especificação da OntoReD também incorporou as orientações da *Ontology 101* (Noy, 2001), que propõe um conjunto de questões a serem respondidas nas consultas que a ontologia deve possibilitar.

2.3 Especificação

A especificação da OntoReD apresenta informações gerais da Ontologia, considerando o domínio, o tipo de ontologia que está inserida, a sua proposta, bem como o escopo e a base de conhecimento que fundamentam a ontologia. O Quadro 1 apresenta o cenário geral da OntoReD.

QUADRO 1 - Especificação da OntoReD

Especificação da Ontologia
Domínio: Dívida de Requisitos
Conceitualização por: Viviane Duarte Bonfim; Fabiane Barreto Vavassori Benitti
Implementação por: Viviane Duarte Bonfim
Proposta: A ontologia OntoReD é uma ontologia de domínio que consiste em manter uma base de informações sobre causas das dívidas de requisitos, consequências das dívidas de requisitos, práticas que podem mitigar as causas e consequentemente suas consequências, as quais são organizadas por meio de áreas da gestão da dívida. O objetivo da OntoReD é aproximar a base de informações de profissionais de organizações ágeis e permitir uma melhor compreensão das causas das dívidas de requisitos e suas consequências considerando seu cenário atual e, a partir disso, ter acesso às recomendações de práticas que poderão mitigar as causas de suas dívidas de requisitos.
Escopo: Os termos que fundamentam a ontologia são apoiados no resultado dos estudos mencionados nas fontes de conhecimento.
Fonte de Conhecimento: Unificação do conhecimento obtido por meio da visão prática, realizada com apoio de profissionais da área de desenvolvimento de organizações ágeis e da literatura por meio de mapeamento sistemático da literatura e estudo terciário (capítulos 3 e 4 da tese de Bonfim; Benitti, 2026).
Questões que a ontologia deve responder: Q1. Quais causas de dívida resultam em uma determinada consequência? Q2. Quais consequências são causadas por uma determinada causa? Q3. Quais são as práticas que podem mitigar as causas de dívida de requisitos? Q4. Quais são as áreas que permitem o gerenciamento de requisitos de dívida? Q5. Quais práticas estão associadas a uma área específica de gerenciamento de dívida? Q6. Qual área de gerenciamento de dívida está associada a determinadas práticas que podem mitigar uma certa causa?

Fonte: Adaptado de Fernández-López, Gómez-Pérez, Juristo (1997)

2.4 Conceitualização

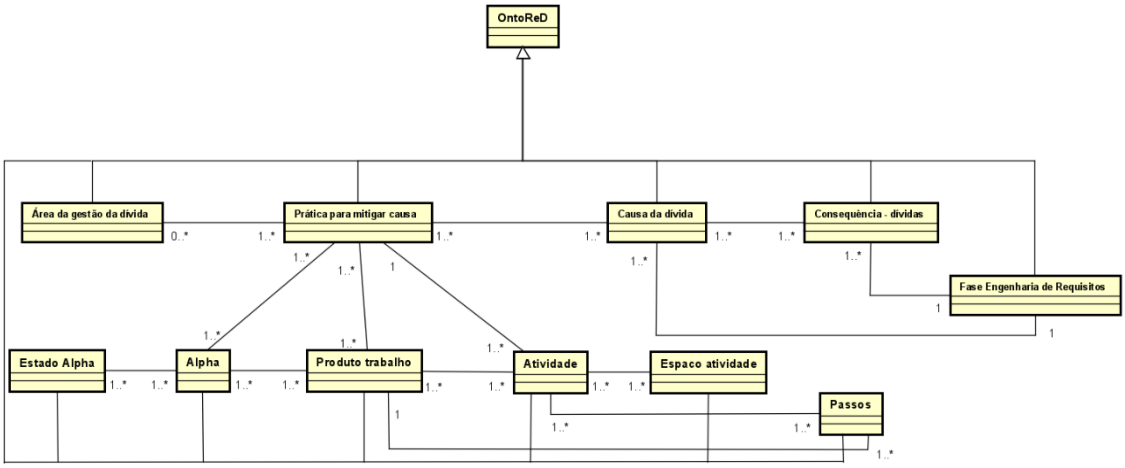
A etapa de conceitualização é destinada à organização e estruturação das informações amparadas pela aquisição do conhecimento obtido por meio das fontes mencionadas na especificação e evidenciado pelo modelo conceitual.

Visando atender o modelo conceitual, Fernández, Gómes-Pérez e Juristo (1997) sugerem elaborar um glossário com os termos relevantes, considerando os conceitos, suas relações, propriedades das instâncias e propriedades envolvendo o domínio da ontologia, o qual é apresentado na Seção 2.9, Quadro 9.

A Figura 1 retrata a associação entre os conceitos representados pelas classes que estão presentes na OntoReD. Na conceitualização das classes procurou-se preservar os termos obtidos nos estudos realizados que deram origem a eles.

O modelo conceitual, Figura 1, é transformado, posteriormente em um modelo formalizado que foi implementado por meio da linguagem da Ontologia (Goméz-Pérez; López; Corcho, 2004).

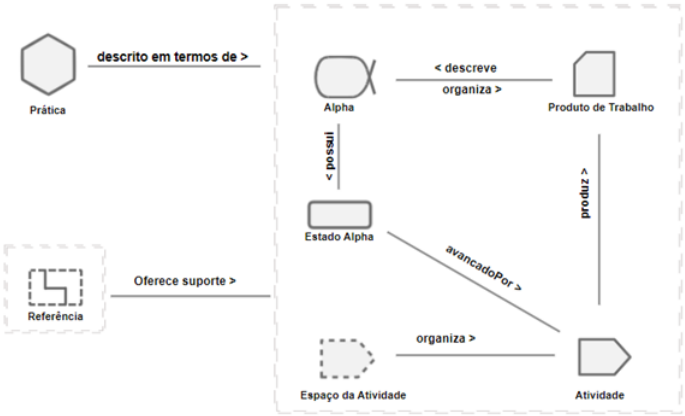
FIGURA 1 - Estrutura das Classes da OntoReD



Fonte: Elaborado pelo autor (2023)

A classe “Prática para mitigar causa” possui um detalhamento que segue uma abordagem constituída de alguns elementos inspirados na Biblioteca de Ivar Jacobson (2018) apoiados pela linguagem Essence (OMG, 2018). A estrutura da prática está representada na Figura 2 e são compostos pelos elementos apresentados no Quadro 2.

FIGURA 2 - Estrutura dos elementos da prática



Fonte: Elaborado pelo autor (2023)

QUADRO 2 - Elementos da Prática conforme Linguagem Essence (OMG, 2018)

Glossário dos elementos da prática		
Elemento	**Ícone**	**Descrição**
Ícone	-	Imagem que representa o elemento.
Identificação do elemento (nome) da Prática		Um elemento pode ser um produto de trabalho, atividade, atividade, alpha ou estado alpha.
Prática para mitigar a dívida		"Uma prática é uma abordagem repetível para fazer algo com um objetivo específico em mente. Uma prática descreve como lidar com um aspecto específico de um empreendimento de engenharia de software, incluindo as descrições de todos os elementos relevantes necessários para expressar a orientação de trabalho desejado para atingir o objetivo da prática. Uma prática pode ser definida como uma composição de outras práticas" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.
Produto de trabalho		"Um produto de trabalho é um artefato de valor e relevância para um empreendimento de engenharia de software. Um produto de trabalho pode ser um documento ou um pedaço de software, ou outras entidades criadas." (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.
Atividade do produto de trabalho		"Uma atividade define uma ou mais abordagens para executar algum trabalho a ser executado e pode recomendar ações sobre alfas e/ou produtos de trabalho para executar esse trabalho." (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.
Espaço atividade		"Um espaço reservado para algo a ser feito no empreendimento de engenharia de software." (Jacobson, 2018). Disponível em:

		https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.
Alpha		"Um alfa representa e mantém o estado de algum elemento, aspecto ou abstração em um empreendimento que tem um estado discernível e o conhecimento de cujo estado é necessário para entender o estado de progresso e/ou saúde do empreendimento" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.
Estado Alpha		"Um estado alfa é uma especificação do estado de progresso de um alfa. O estado de um alfa é determinado pela avaliação de itens da lista de verificação" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.

Fonte: Adaptado da Linguagem Essence (OMG 2018)

2.5 Conceitualização – Detalhamento dos elementos das práticas específicas

As práticas adotadas seguem o conjunto de elementos definidos pela Linguagem Essence (OMG, 2018), conforme apresentado na Seção 2.4. A estrutura, a organização, as informações e as propriedades utilizadas no detalhamento dessas práticas são ilustradas por meio das práticas *"Elaborar artefato da UML – Diagrama de caso de uso" e "Requisitos bem definidos registrados no backlog"*, que exemplificam o nível de descrição esperado. O relacionamento entre os elementos dessas práticas pode ser observado na Seção 2.6.2, Quadro 4, considerando, de modo geral, as propriedades estabelecidas entre "*domain*" e "*range*".

2.5.1 Elaborar diagrama de caso de uso e Requisitos bem definidos registrados no backlog

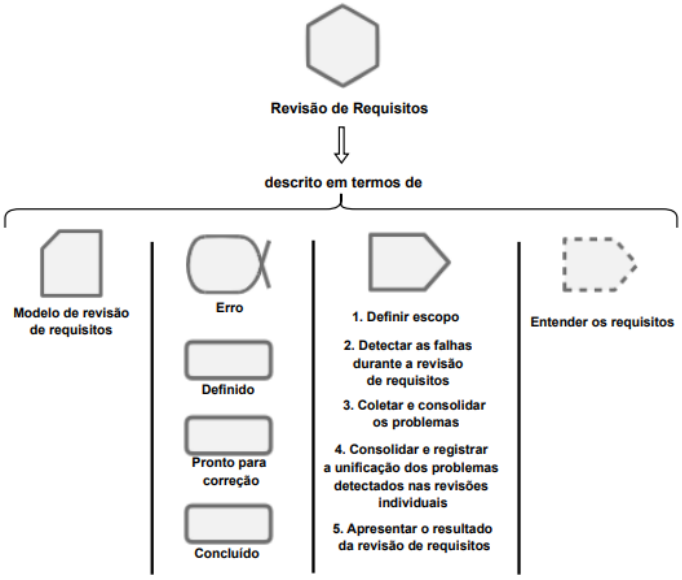
O detalhamento dos elementos das práticas "*Elaborar artefato da UML - Diagrama de caso de uso*" e "*Requisitos bem definidos registrados no backlog*", bem como a notação utilizada foram apoiados por meio da biblioteca de práticas[1] (Jacobson, 2018) e estão implementados diretamente na Ontologia.

[1] A biblioteca de práticas está passando por atualização conforme informado em https://www.ivarjacobson.com/practice-library-beta, proposta por Jacobson (2018).

2.5.2 Prática: Revisão de requisitos

Para apresentar o detalhamento da prática *"Revisão de Requisitos"*, é apresentada a estrutura, organização e informações dos elementos da prática, por meio da Figura 3.

FIGURA 3 - Elementos da prática Revisão de Requisitos



Fonte: Elaborado pelo autor (2024)

2.5.2.1 Produto de trabalho: Modelo de Revisão de requisitos

O Modelo de revisão de requisitos consiste de um único produto de trabalho pertencente à prática *"Revisão de Requisitos"* e possui os vínculos com elementos mapeados na ontologia por meio de propriedades entre *"domain"* e *"range"* (Seção 2.6.2).

No modelo de revisão de requisitos são contempladas as seguintes informações (propriedades dos dados):

- **<u>Apresentação:</u>**
- Prática que permite a revisão de requisitos em diferentes artefatos das Atividades da Engenharia de Requisitos.
- **<u>Descrição:</u>** A revisão de requisitos é composta por um grupo de revisores (Washizaki, 2024), que valida os requisitos e é formada por engenheiros de software, clientes, usuários e outros envolvidos que examinam a especificação em busca de erros no conteúdo ou na interpretação, de áreas que talvez sejam necessários esclarecimentos, de informações ausentes, de inconsistências, de requisitos conflitantes (Pressman; Maxim, 2021). O artefato a ser

revisado pode ser um documento, um diagrama, um protótipo ou qualquer outro artefato que especifique requisitos.

- **Níveis de detalhes - (OntoReD está definido como: Passos elementos da prática):**

Os níveis de detalhes possuem o vínculo *"estaContidoEm"* com o modelo de revisão de requisitos e possui os seguintes níveis:

- **Planejamento e preparação da revisão de requisitos:** O primeiro passo para realizar a revisão de requisitos é definir, neste nível:
  - Os envolvidos na revisão, ou seja, os revisores que revisarão o artefato.
  - O projeto no qual o artefato que será revisado está vinculado.
  - O artefato que será revisado e sua versão.
  - O objetivo da revisão.
  - A previsão da data da revisão.
  - A estimativa de esforço que será dedicada à revisão.
- **Execução da revisão de requisitos:** No nível de execução, os revisores procuram por falhas nos requisitos, apoiadas no artefato que está sendo revisado, por exemplo, diagrama de caso de uso, documento de especificação de requisitos, etc.

A revisão pode ser realizada individualmente por cada revisor, parte dos revisores ou pelo time todo.

Durante o nível de execução, todas as falhas devem ser registradas para serem corrigidas posteriormente e, para isso, recomenda-se registrar as seguintes informações:

- Nome do revisor e seu papel.
- Artefato revisado, sua versão no projeto ao qual está vinculado e versão do projeto.
- Registro de falhas: Para cada falha, informar a descrição da falha, a localização da falha no artefato, a página, quando houver, e o tipo de falha - omissão, ambiguidade...
- Informar as dependências: Por exemplo: Se o artefato revisado for um diagrama de caso de uso, pode informar as dependências ou referências daquele caso de uso como os requisitos funcionais e não funcionais associados a ele.

- Data da revisão.
- Estimativa de esforço prevista inicialmente para a revisão e o esforço real dedicado à revisão.
    - O esforço pode ser apresentado em horas, minutos ou dia, porém, é preciso padronizar.

- **Fechamento da revisão de requisitos:** No nível de fechamento da revisão de requisitos, os revisores se reúnem para compartilharem e analisarem suas revisões observando os registros individuais de problemas - semelhantes ou não, para unificá-los posteriormente. Após essa conferência, os revisores sintetizam as falhas em um único registro e simplificam as redundâncias de registros dos problemas identificados nas revisões individuais. Para o registro de fechamento, recomenda-se manter as seguintes informações, destacadas na execução:
    - Nome e papel dos revisores.
    - Artefato revisado, sua versão e o projeto ao qual está vinculado e versão.
    - Registro de problema: Para cada problema, informar a descrição do problema, a localização do problema no artefato, a página, quando houver e o tipo de falha - omissão, ambiguidade…
    - Informar as dependências: Por exemplo: Se o artefato revisado for um diagrama de caso de uso, pode informar as dependências ou referências daquele caso de uso como os requisitos funcionais e não funcionais associados a ele.
    - Medidas corretivas para os problemas identificados.
    - Data do fechamento da revisão de requisitos.
    - Estimativa de esforço total considerando os revisores envolvidos e o esforço real.
        - O esforço pode ser apresentado em horas, minutos ou dia, porém, é preciso padronizar.
- **Apresentação dos resultados da revisão:** Neste nível os revisores compartilham com a equipe do projeto e, caso necessário, também com os stakeholders, os resultados da revisão de requisitos com as informações

sintetizadas no nível de fechamento da revisão de requisitos para adequações e encaminhamentos.

- Após as adequações, se for relevante, poderá realizar uma nova revisão dos requisitos do artefato já revisado.

2.5.2.2 Alpha: Erro

Erro caracteriza-se como um problema em produtos de trabalho voltado aos requisitos, como o que se encontra em um documento de especificação de requisitos durante a revisão de requisitos, por exemplo.

Um erro pode ser categorizado em (Geraldi; Oliveira, 2017)**:**

- **De omissão:** qualquer informação necessária que tenha sido omitida.
- **Fato Incorreto:** informação contraditória com o conhecimento que se tem no domínio de aplicação.
- **Inconsistência:** informação que consta de forma repetitiva, escrita de forma diferente.
- **Ambiguidade:** quando a informação pode levar a múltiplas interpretações.
- **Informação Estranha:** qualquer informação que, embora relacionada ao domínio, não é necessária para o sistema.
- **Outros:** qualquer outro tipo de erro que não se encaixe nas outras categorias. Exemplo: declarações em seções erradas.

2.5.2.3 Estado Alpha: Erro

Os estados alpha referente ao alpha “erro” caracterizam-se como:

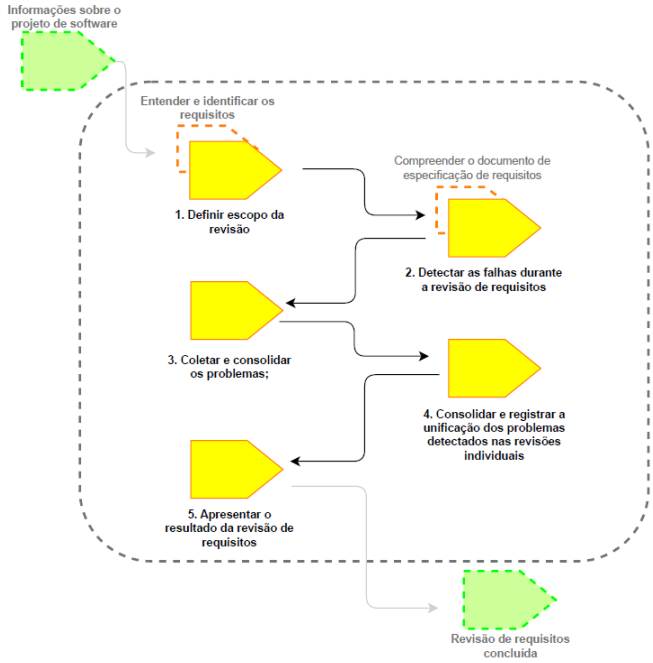
- **Definido:** Definida a categorização dos erros detectados no artefato revisado durante o processo de requisitos.
  - Existe uma compreensão compartilhada da classificação do erro detectado e seu contexto.
  - Para cada erro identificado, deve ser informada, além da identificação e descrição, a categoria do erro e a prioridade para correção, a qual é acordada com os stakeholders.
  - Deve-se manter uma recomendação de solução para cada erro detectado.

- **Pronto para correção:** Os erros detectados e unificados pelos revisores são encaminhados para correção.
    - Existe um profissional destinado para correção dos erros.
    - A descrição do erro é suficientemente compreendida pelo responsável pela correção.
- **Concluído:** O erro foi corrigido e o artefato correspondente foi atualizado.

2.5.2.4 Atividade do produto de trabalho

A sequência das atividades da prática *"Revisão dos Requisitos"* é apresentada na Figura 4.

FIGURA 4 - Sequência das atividades da prática



Fonte: Elaborado pelo autor (2024)

As descrição das atividades consistem em:

1. **Definir escopo da revisão:** Definir envolvidos e seus papéis, item para revisão, versão, projeto vinculado, objetivo.
2. **Detectar as falhas durante a revisão de requisitos:** A detecção de problemas é realizada pelos revisores por meio da varredura no item que está sendo revisado e pode ser

conduzida individualmente por cada revisor ou conjuntamente pelos revisores. As falhas encontradas devem ser documentadas sistematicamente para posterior correção (Pohl; Rupp, 2015).

3. Coletar e consolidar os problemas: Durante a coleta de falhas, todas as falhas identificadas devem ser coletadas, consolidadas e registradas. O momento de consolidação sinaliza falhas que foram detectadas várias vezes, assim como falhas que não caracterizam erro. Este último caso pode acontecer quando um revisor interpretar de forma equivocada um requisito ou alguma restrição sobre ele. Na consolidação também devem ser registradas, além das falhas e erros, as medidas corretivas sugeridas (Pohl; Rupp, 2015).

4. Consolidar e registrar a unificação dos problemas detectados nas revisões individuais: Após a coleta e consolidação dos problemas, os revisores devem se reunir, verificar todas as falhas detectadas individualmente, eliminar as redundâncias nos registros das falhas, agrupar falhas semelhantes que se complementam e construir uma nova lista de falhas unificadas.

5. Corrigir os erros, após a unificação: Os erros devem ser corrigidos e sua correção, registrada.

6. Apresentar o resultado da revisão de requisitos: Finalizada a unificação da lista de problemas, bem como a sua correção, o registro da revisão deve ser apresentado aos envolvidos e convidados, disponibilizando-a, posteriormente, para o time.

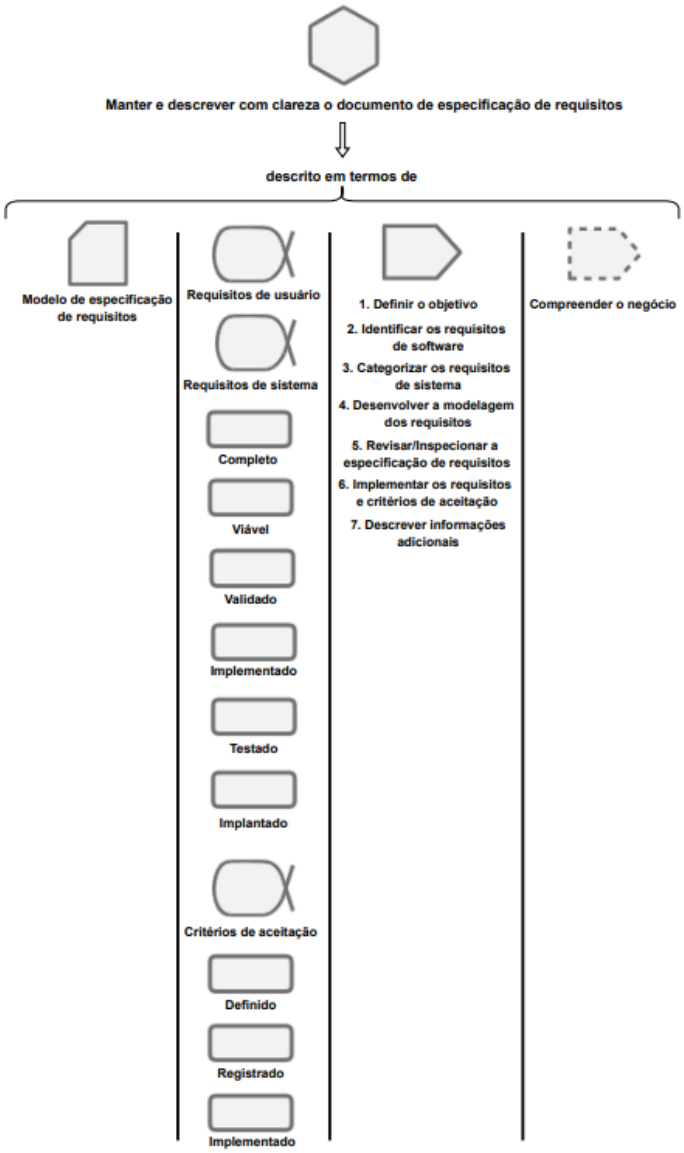
2.5.2.5 Espaço Atividade

- São caracterizados como:
  - **Entender os requisitos de software:** Utiliza as informações da prática de documento de requisitos.
  - **Compreender o documento de especificação de requisitos:** Utiliza as informações da prática de documento de requisitos.

2.5.3 Prática: Manter e descrever com clareza o documento de especificação de requisitos

Para apresentar o detalhamento da prática *"Manter e descrever com clareza o documento de especificação de requisitos"*, é apresentada a estrutura, organização e informações dos elementos da prática na Figura 5 .

FIGURA 5 - Elementos da prática - Manter e descrever com clareza o documento de especificação de requisitos[2]



Fonte: Elaborado pelo autor (2024)

2.5.3.1 Produto de trabalho: Modelo de especificação de requisitos

No modelo de especificação de requisitos são contempladas as seguintes informações (propriedades dos dados):

- **Apresentação:** Em engenharia de software, *"especificação de requisitos de software"* normalmente se refere à produção de um documento que pode ser sistematicamente revisado, avaliado e aprovado.

[2] Os elementos alpha "requisitos de usuário e "requisitos de sistema" possuem os mesmos estados alpha.

- **Descrição:** A especificação de requisitos de software permite uma avaliação rigorosa dos requisitos antes do projeto de design, reduzindo a necessidade de retrabalho e de redesenho posteriores. Ela também deve fornecer uma base realista para estimar custos, riscos e cronogramas do produto. **As organizações também podem usar um documento de especificação de requisitos de software como base para desenvolver planos eficazes de verificação e validação.**
- **Níveis de detalhes - (OntoReD está definido como: Passos elementos da prática):** Os níveis de detalhes possuem os seguintes níveis:
    - **Valor estabelecido:** Para realizar a especificação de requisitos recomenda-se definir e registrar informações organizando-as por meio de um documento de especificação de requisitos, o qual pode ser amparado em um padrão IEEE para documentos de requisitos (IEEE, 2018), pois é um padrão genérico e pode ser adaptado para usos específicos.
        - Prefácio: deve definir o público esperado do documento e descrever seu histórico de versões, incluindo uma justificativa para a criação de uma nova versão e um resumo das alterações feitas em cada versão.
        - Introdução: Na introdução, recomenda-se descrever a necessidade do sistema. Ele deve descrever brevemente as funções do sistema e explicar como ele funcionará com outros sistemas. Sugere-se também, descrever como o sistema se encaixa nos objetivos gerais de negócios ou estratégicos da organização que está requerendo o software.
        - Glossário: deve definir os termos técnicos usados no documento. Não se deve expor suposições sobre a experiência ou conhecimento do leitor.
        - Definição de requisitos de usuário: Descreve-se os serviços fornecidos para o usuário. Os requisitos não funcionais do sistema também devem ser descritos nesta seção. Esta descrição pode usar linguagem natural, diagramas ou outras notações que sejam compreensíveis para os clientes. Os padrões de produto e processo que devem ser seguidos devem ser especificados.
        - Especificação de requisitos de sistemas: Descrever os requisitos funcionais e não funcionais em mais detalhes. Se necessário, outros detalhes também podem ser adicionados aos requisitos não funcionais. Interfaces para outros sistemas podem ser acrescentadas.

- Modelos de sistemas: Pode incluir modelos gráficos de sistema mostrando os relacionamentos entre os componentes do sistema, o sistema e seu ambiente. Exemplos de possíveis modelos são modelos de objetos, modelos de fluxo de dados ou modelos de dados semânticos.
- Arquitetura de sistema: Este item deve apresentar uma visão geral de alto nível da arquitetura do sistema antecipada, mostrando a distribuição de funções entre os módulos do sistema. Os componentes arquitetônicos que são reutilizados devem ser destacados.
- Evolução do sistema: Descrever as suposições fundamentais nas quais o sistema é baseado e quaisquer mudanças antecipadas devido à evolução do hardware, mudanças nas necessidades do usuário e assim por diante. Esta seção é útil para designers de sistemas, pois pode ajudá-los a evitar decisões de design que restringiriam prováveis mudanças futuras no sistema.
- Apêndices: Fornecer informações detalhadas e específicas relacionadas ao sistema que está sendo desenvolvido. Por exemplo: descrições de hardware e banco de dados. Os requisitos de hardware definem as configurações mínimas e ideais para o sistema. Os requisitos de banco de dados definem a organização lógica dos dados usados pelo sistema e os relacionamentos entre os dados.
- Índice: Vários índices para o documento podem ser incluídos. Além de um índice alfabético normal, pode haver um índice de diagramas, um índice de funções e assim por diante.

- **Limite do sistema estabelecido:** Os requisitos do usuário são quase sempre escritos em linguagem natural, suplementados por diagramas e tabelas apropriados no documento de requisitos, os quais podem seguir a orientação de histórias de usuário.
    - **Estrutura recomendada para história de usuário: Quem? O que? Para que?:** Os requisitos do sistema também podem ser escritos em linguagem natural, mas outras notações baseadas em formulários, modelos de sistema gráfico ou modelos de sistema matemático também podem ser usadas. **Possíveis notações que podem ser usadas para escrever requisitos do sistema:**

- **Frases em linguagem natural:** Os requisitos são escritos usando frases numeradas em linguagem natural. Cada frase deve expressar um requisito.
- **Linguagem natural estruturada:** Os requisitos são escritos em linguagem natural em um formulário ou modelo padrão. Cada campo fornece informações sobre um aspecto do requisito.
- **Linguagens de descrição de design:** Esta abordagem usa uma linguagem como uma linguagem de programação, mas com recursos mais abstratos para especificar os requisitos definindo um modelo operacional do sistema. Esta abordagem é raramente usada, embora possa ser útil para especificações de interface.
- **Notações gráficas:** Modelos gráficos complementados por anotações de texto, são usados para definir os requisitos funcionais para o sistema; diagramas da UML (Linguagem de Modelagem Unificada), como diagramas de caso de uso são comumente usados.
- **Especificações matemáticas:** Essas notações são baseadas em conceitos matemáticos como máquinas ou conjuntos de estado finito. Embora essas especificações inequívocas possam reduzir a ambiguidade em um documento de requisitos, a maioria dos clientes não entende uma especificação formal. Eles não podem verificar se ela representa o que eles querem e relutam em aceitá-la como um contrato de sistema.
    - Modelos gráficos são mais úteis quando você precisa mostrar como um estado muda ou quando é preciso descrever uma sequência de ações.
    - Os diagramas de sequência e de estado da UML mostram a sequência de ações que ocorrem em resposta a uma determinada mensagem ou evento.
    - Especificações matemáticas formais são algumas vezes usadas para descrever os requisitos para sistemas críticos de segurança, mas são raramente usadas em outras circunstâncias.

- **Recomendações: Para evitar problemas de ambiguidade, requisitos vagos, Sommerville (2018) recomenda as seguintes diretrizes:**
- **<u>Linguagem natural:</u>**
    - Estabelecer um formato padrão e garantir que todas as definições de requisitos sigam esse formato.
        - Padronizar o formato torna as omissões menos prováveis e os requisitos mais fáceis de verificar.
        - O formato deve expressar o requisito em uma única frase.
        - Associa-se uma declaração de justificativa a cada requisito do usuário para explicar porquê o requisito foi proposto.
        - A justificativa também pode incluir informações sobre quem propôs o requisito (a fonte do requisito) para identificar quem consultar se o requisito tiver que ser alterado.
    - Adotar a linguagem consistentemente para distinguir entre requisitos obrigatórios e desejáveis.
        - Requisitos obrigatórios são requisitos que o sistema deve suportar e geralmente são escritos usando ‘deve’.
        - Requisitos desejáveis não são essenciais e são escritos usando ‘pode’.
    - Usar destaque de texto (negrito, itálico ou colorido) para destacar partes-chave do requisito.
    - Não presumir que os leitores entendam a linguagem técnica de engenharia de software.
        - É fácil que palavras como “arquitetura” e “módulo” sejam mal interpretadas.
        - Deve-se, portanto, evitar o uso de jargões, abreviações e acrônimos.
    - Sempre que possível, associar uma justificativa a cada requisito do usuário.

        - A justificativa deve explicar porquê o requisito foi incluído.
        - É particularmente útil quando os requisitos são alterados, pois pode ajudar a decidir quais mudanças seriam indesejáveis.

- **<u>Linguagem natural estruturada:</u>**

    - Linguagem natural estruturada é uma maneira de escrever requisitos de sistema onde a liberdade do escritor de requisitos é limitada e todos os requisitos são escritos de uma maneira padrão.
    - Essa abordagem mantém a maior parte da expressividade e compreensibilidade da linguagem natural, mas garante que alguma uniformidade seja imposta à especificação.
    - As notações de linguagem estruturada usam modelos para especificar os requisitos do sistema.
    - Recomenda-se que os requisitos do usuário sejam inicialmente escritos em cartões, sendo um requisito por cartão.
        - Eles sugerem uma série de campos em cada cartão, como a justificativa dos requisitos, as dependências de outros requisitos, a fonte dos requisitos, materiais de suporte e assim por diante.

- **Sugestão de especificação padrão para especificar requisitos funcionais:**
    - Uma descrição da função ou entidade sendo especificada.
    - Uma descrição de suas entradas e de onde elas vêm.
    - Uma descrição de suas saídas e para onde elas vão.
    - Informações sobre as informações que são necessárias para a computação ou outras entidades no sistema que são usadas (a parte "requer").
    - Uma descrição da ação a ser especificada.
    - Se uma abordagem funcional for usada, uma pré-condição definindo o que deve ser verdadeiro antes que a função seja

chamada, e uma pós-condição especificando o que é verdadeiro depois que a função for chamada.

- Uma descrição dos efeitos colaterais (se houver) da operação.
- O uso de especificações estruturadas remove alguns dos problemas da especificação de linguagem natural.

A variabilidade na especificação é reduzida e os requisitos são organizados de forma mais eficaz. No entanto, ainda é difícil escrever requisitos de forma clara e inequívoca, particularmente quando cálculos complexos (por exemplo, como calcular a dose de insulina) devem ser especificados.

Para resolver esse problema, pode-se adicionar informações extras aos requisitos de linguagem natural, por exemplo, usando tabelas ou modelos gráficos do sistema.

2.5.3.2 Alpha: Requisitos de usuário

Os requisitos de usuário, definição na Figura 6, são declarações descritas sob a perspectiva do usuário, em linguagem natural adicionada a diagramas, dos serviços que se espera que o sistema ofereça aos seus usuários e das limitações sob as quais ele pode operar.

FIGURA 6 - Definição de requisitos de usuário

Definição de requisitos de usuário

1. O sistema Mentcare deve gerar relatórios de gestão mensais, mostrando o custo dos medicamentos prescritos por cada clínica naquele mês.

Fonte: Sommerville, 2018

Esses requisitos podem variar de declarações abrangentes das características necessárias do sistema até descrições detalhadas da sua funcionalidade.

- Para registrá-los, sugere-se o uso de histórias de usuário (HU), seguindo o padrão que responde aos seguintes elementos: **Quem? O que? Para que?**
- A partir desses registros deve ser possível extrair os requisitos de sistemas. O registro deve ser suficientemente compreensível pelos usuários do sistema que não possuem conhecimento técnico detalhado.

Exemplo de HU1: "*Enquanto Luiz, eu gostaria de ter a possibilidade de criar instâncias de MonogDB na Umbler para usar em meus projetos web*" (Duarte, 2021).

Recomendação para descrição da HU (Duarte, 2021)**:**

- A HU deve ser desconsiderar detalhes de implementação.
- Focada no que o usuário deseja fazer.
- Descrita sob o ponto de vista do usuário.
- Curta e objetiva.

2.5.3.3 Estado Alpha: Requisitos de usuário

Os estados alpha de requisitos de usuário são descritos como segue:

- **Definido:** Os requisitos de usuário foram identificados e definidos.
- **Registrado:** Os requisitos de usuário foram documentados.
- **Implementado:** Os requisitos de usuário foram implementados e atendidos.

2.5.3.4 Alpha: Requisitos de sistema

Os requisitos de sistema são descrições mais detalhadas das funções, dos serviços e das restrições operacionais do sistema de software.

O documento de requisitos de sistema (especificação funcional) deve definir exatamente o que deve ser implementado. Eles podem ser usados como parte do contrato para a implementação do sistema e deve, portanto, ser uma especificação completa e detalhada de todo o sistema.

Idealmente, os requisitos do sistema simplesmente descrevem o comportamento externo do sistema e suas restrições operacionais. Eles não devem se preocupar em como o sistema deve ser projetado ou implementado.

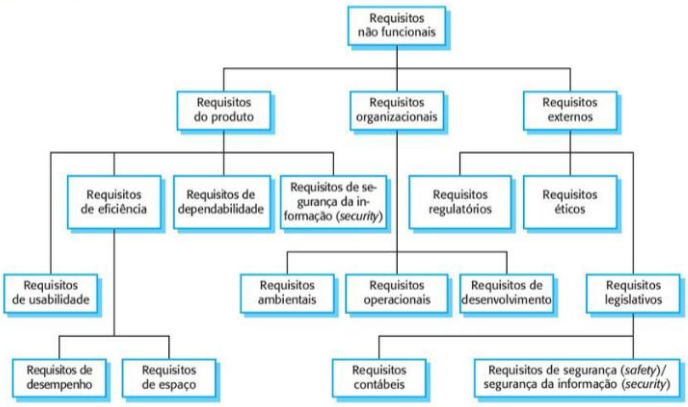
Os requisitos de sistema podem ser classificados em (Sommerville, 2018)**:**

- **Requisitos funcionais:** São declarações de serviços que o sistema deve fornecer, como o sistema deve reagir a entradas específicas.
    - Exemplos de requisitos funcionais:
        - Uma secretária deve fazer uma busca na lista de consultas de todas as clínicas.
        - O sistema deve gerar para a secretária, a cada dia e para cada clínica, uma lista de pacientes que devem comparecer às consultas naquele dia.
- **Requisitos não funcionais ou de qualidade:** São restrições aos serviços ou funções oferecidos pelo sistema e sua classificação pode ser visualizada na Figura 7 e exemplos são demonstrados na Figura 8.

- Eles incluem restrições de tempo, restrições ao processo de desenvolvimento e restrições impostas por padrões.
- Requisitos não funcionais geralmente se aplicam ao sistema como um todo, em vez de recursos ou serviços individuais do sistema.

- **Os requisitos não funcionais ou de qualidade podem ser classificados como** (Sommerville, 2018)**:**
  - **Requisitos do produto:** Requisitos que especificam ou restringem o comportamento do software.
    - Exemplos incluem requisitos de desempenho sobre a rapidez com que o sistema deve executar e quanta memória ele requer, requisitos de confiabilidade que definem a taxa de falha aceitável, requisitos de segurança e requisitos de usabilidade.
  - **Requisitos organizacionais:** São requisitos amplos do sistema derivados de políticas e procedimentos na organização do cliente e do desenvolvedor.
    - Exemplos incluem requisitos de processo operacional que definem como o sistema será usado, requisitos de processo de desenvolvimento que especificam a linguagem de programação, o ambiente de desenvolvimento ou padrões de processo a serem usados e requisitos ambientais que especificam o ambiente operacional do sistema.
  - **Requisitos externos:** Refere-se aos requisitos que são derivados de fatores externos ao sistema e seu processo de desenvolvimento.
    - Podem incluir requisitos regulatórios que definem o que deve ser feito para que o sistema seja aprovado para uso por um regulador, como um banco central.
    - Requisitos legislativos que devem ser seguidos para garantir que o sistema opere dentro da lei.
    - Requisitos éticos que garantam que o sistema será aceitável para seus usuários e o público em geral.

FIGURA 7 - Classificação de requisitos de qualidade (não funcionais)



Fonte: Sommerville, 2018

FIGURA 8 - Exemplos de requisitos de qualidade

Requisito do produto

O sistema Mentcare deve ficar disponível para todas as clínicas durante o expediente normal (segunda-sexta, 8h30-17h30).

O tempo que o sistema pode permanecer fora do ar no expediente normal não deve ultrapassar 5 segundos em qualquer dia.

Requisito organizacional

Os usuários do sistema Mentcare devem se identificar usando o cartão de identificação de autoridade de saúde.

Requisito externo

O sistema deve implementar providências para a privacidade do paciente, conforme estabelecido em HStan-03-2006-priv.

Fonte: Sommerville, 2018

2.5.3.5 Estado Alpha: Requisitos de sistema

Os estados alpha de requisitos de sistema são descritos como segue:

- **Definido:** Os requisitos de sistema foram identificados e definidos.
- **Categorizado:** Os requisitos de sistema foram categorizados em: Requisitos funcionais e de qualidade.
- **Registrado:** Os requisitos de sistema foram documentados.
- **Implementado:** Os requisitos de sistema foram implementados e atendidos.

2.5.3.6 Alpha: Critérios de aceitação

Os critérios de aceitação são registros sobre regras de negócio ou requisitos de qualidade pertencentes a uma história de usuário, ou seja, um checklist de requisitos que a história de

usuário, em particular, deve atender para ser sinalizada como “done” (finalizada) pelo *Product Owner* (Duarte, 2021).

- **Exemplos de critérios de aceitação HU1** (Duarte, 2021):
    - A senha do banco do banco MongoDB deve ter, no mínimo, 08 caracteres alfanuméricos.
    - A tela deve ser desenvolvida conforme o layout sugerido pelo time de IU_UX.
    - O nome do banco do MongoDB criado pelo usuário não pode ser igual a outro já existente.

2.5.3.7 Estado Alpha: Critérios de aceitação

Os estados alpha referente aos critérios de aceitação são descritos como segue:

- **Definido:** Os critérios de aceitação foram definidos.
- **Registrado:** Os critérios de aceitação foram registrados e vinculados ao requisito, o qual está associado.
- **Implementado:** Os critérios de aceitação foram implementados e atendidos.

2.5.3.8 Atividades

As atividades da prática *“Manter e descrever com clareza o documento de especificação de requisitos” são:*

1. **Descrever o objetivo do software:** O objetivo do software deve ser descrito textualmente abordando os aspectos fundamentais que justificam o seu desenvolvimento.
    - Recomenda-se utilizar uma linguagem clara e não técnica.

2. **Identificar os requisitos de software:** usuário e sistema:

Os requisitos de software são frequentemente escritos em linguagem natural, mas, na especificação de requisitos de software, isso pode ser complementado por descrições formais ou semiformais.

A seleção de notações apropriadas permite que requisitos específicos e aspectos da arquitetura de software sejam descritos de forma mais precisa e concisa do que a linguagem natural.

- **Os requisitos de usuário** devem ser identificados e registrados sob essa perspectiva.
- **Os requisitos do sistema** são versões expandidas dos requisitos do usuário e incluem detalhes e explicam como os requisitos do usuário devem ser fornecidos pelo sistema.
    - Eles podem ser usados como parte do contrato para a implementação do sistema e devem, portanto, ser uma especificação completa e detalhada de todo o sistema.

3. **Categorizar os requisitos de sistema:** Para registrar os requisitos de sistema, recomenda-se categorizá-los em requisitos funcionais e requisitos de qualidade ou não funcionais.

- **Os requisitos funcionais** devem representar "o que o software deve atender", ou seja, as funcionalidades do software.
- **Os requisitos de qualidade**, também conhecidos como requisitos não funcionais, representam as restrições do software como, restrições direcionadas ao desempenho, confiabilidade e usabilidade, por exemplo.

Para registrar requisitos funcionais (RF) e de qualidade (RQ), recomenda-se utilizar um padrão para o registro a fim de facilitar a compreensão e o mesmo vocabulário pelos membros do time.

Recomenda-se incluir:

- Identificação do requisito funcional, denominação e status (obrigatório/desejável): RF 01 - Manter agendamento de consulta médica.
- Descrição do RF e suas informações.
- Identificação e nome do requisito de qualidade vinculado ao RF 01.
- Descrição do(s) requisito(s) de qualidade.

4. **Desenvolver a modelagem dos requisitos:**

A modelagem de requisitos pode ser representada por diferentes artefatos, os quais poderão ser desenvolvidos conforme a necessidade da organização em representar seu projeto de software ou parte dele.

Dentre os artefatos para especificar os requisitos, sugere-se:

- **Descrição dos cenários de casos de uso:** Os casos de uso são uma técnica para captar os requisitos funcionais de um sistema. Exemplo de descrição de cenário é demonstrado na Figura 9.
    - Servem para descrever as interações típicas entre os usuários de um sistema e o próprio sistema, fornecendo uma narrativa sobre como o sistema é utilizado.
    - Um cenário é uma sequência de passos que descreve uma interação entre um usuário e um sistema. Assim, se tivermos uma loja on-line baseada na Web (loja virtual), poderia ter um cenário de Compra de um Produto que descreve:

        O cliente navega no catálogo de itens e adiciona os itens desejados à sua cesta de compras. Quando o cliente deseja pagar, descreve o endereço de entrega, fornece as informações do cartão de crédito e confirma a venda. O sistema verifica a autorização do cartão de crédito e confirma a venda imediatamente e com um e-mail subsequente.

Um ator é um papel que um usuário desempenha com relação ao sistema. Os atores podem ser o cliente, representante de serviço ao cliente, gerente de vendas e analista de produto. Os atores realizam os casos de uso.

- Um ator não precisa ser um ser humano.
- Se o sistema realiza um serviço para outro sistema de computador, esse outro sistema é um ator.

Cada caso de uso tem um ator principal, que pede ao sistema para que execute um serviço. Podem existir outros atores com os quais o sistema se comunica enquanto executa o caso de uso, conhecidos como atores secundários. E, **cada passo em um caso de uso, é um elemento da interação entre um ator e o sistema.**

- Deve ser uma declaração simples mostrando claramente quem está executando o passo.
- O passo deve apresentar a intenção do ator e não os mecanismos do que o ator faz.

- Uma extensão dentro do caso de uso nomeia uma condição que resulta em diferentes interações daquelas descritas no cenário principal de sucesso e informa quais são essas diferenças.
- Iniciar a extensão dando um nome ao passo em que a condição é detectada e fornecer uma breve descrição da condição.
- Após a condição, enumerar os passos, no mesmo estilo que o do cenário principal de sucesso.
- Concluir esses passos descrevendo onde retorna para o cenário principal de sucesso, caso retorne.

FIGURA 9 - Exemplo de descrição de cenário

Compra de um Produto

Nível do Objetivo: Nível do Mar

Cenário Principal de Sucesso:

1. O cliente navega pelo catálogo e seleciona itens para comprar
2. O cliente vai para o caixa
3. O cliente preenche o formulário da remessa (endereço de entrega; opção de entrega imediata ou em três dias)
4. O sistema apresenta a informação completa do faturamento, incluindo a remessa
5. O cliente preenche a informação de cartão de crédito
6. O sistema autoriza a compra
7. O sistema confirma imediatamente a venda
8. O sistema envia uma confirmação para o cliente por *e-mail*

Extensões:

3a: Cliente regular
.1: O sistema mostra a informação atual da remessa, a informação de preço e a informação de cobrança
.2: O cliente pode aceitar ou escrever por cima desses padrões, retornando ao CPS, no passo 6

6a. O sistema falha na autorização da compra a crédito
.1: O cliente pode inserir novamente a informação do cartão de crédito ou cancelar

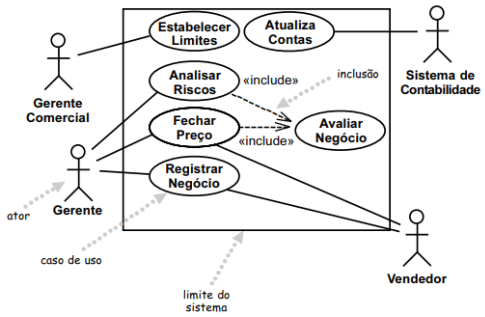
Fonte: Fowler, 2007

Para representar a especificação de requisitos recomenda-se os seguintes artefatos, conforme a Unified Modeling Language™ (UML®):

- **Diagrama de casos de uso:** O diagrama de casos de uso é uma técnica para captar os requisitos funcionais de um sistema. Eles servem para descrever as interações típicas entre os usuários de um sistema e o próprio sistema, fornecendo uma narrativa sobre como o sistema é utilizado. A Figura 10 apresenta um exemplo de diagrama de casos de uso.
    - A melhor maneira de pensar um diagrama de caso de uso é como um sumário gráfico do conjunto de casos de uso.
    - O diagrama de casos de uso mostra os atores, os casos de uso e os relacionamentos entre eles:
        - Quais atores realizam quais casos de uso.
        - Quais casos de uso incluem outros casos de uso.

- Os casos de uso são uma ferramenta valiosa para ajudar no entendimento dos requisitos funcionais de um sistema.
- Uma primeira passagem nos casos de uso deve ser feita no início.
- Versões mais detalhadas dos casos de uso devem ser elaboradas apenas antes do desenvolvimento desse caso de uso.

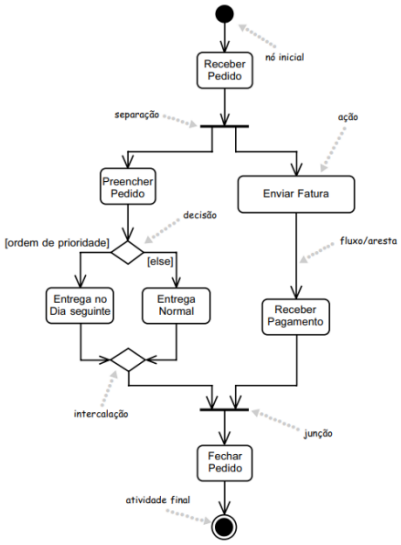
FIGURA 10 - Exemplo de diagrama de caso de uso



Fonte: Fowler, 2007

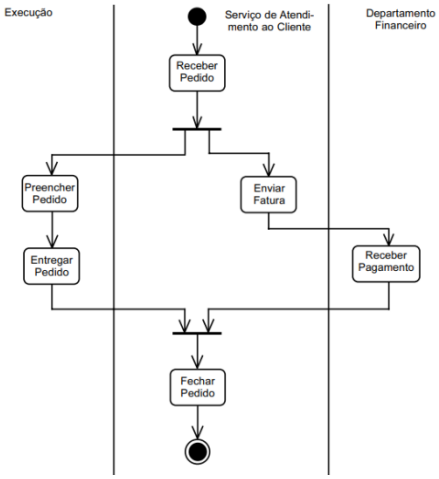
- **Diagrama de atividades:** Os diagramas de atividades são uma técnica para descrever a lógica de procedimento, processo de negócio e fluxo de trabalho. As Figuras 11 e 12 demonstram exemplos de diagrama de atividades.
  - O diagrama de atividades permite que quem está seguindo o processo escolha a ordem na qual fazer as coisas.
  - Ele simplesmente determina as regras essenciais de sequência que se deve seguir.
    - Isso é importante para modelagem de negócios, pois os processos frequentemente ocorrem em paralelo.
  - A maior qualidade dos diagramas de atividades está no fato de que eles suportam e estimulam o comportamento paralelo. Isso os torna uma excelente ferramenta para modelagem de fluxos de trabalho e de processos.

FIGURA 11 - Exemplo 1 de diagrama de atividade



Fonte: Fowler, 2007

FIGURA 12 - Exemplo 2 de diagrama de atividade

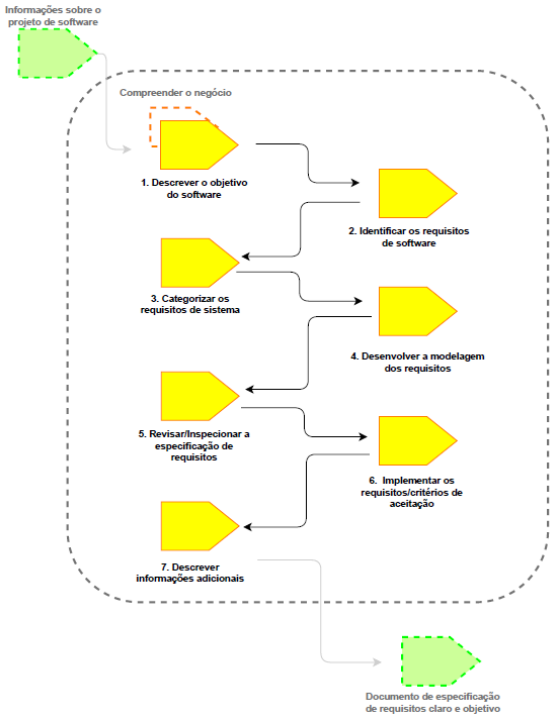


Fonte: Fowler, 2007

5. **Revisar/Inspecionar a especificação de requisitos:** Orientações disponíveis em: Usar um link para a parte de revisão.
6. **Descrever informações adicionais:** Incluir informações que julgar relevantes.

A Figura 9 apresenta o fluxo das atividades da prática *"Manter e descrever com clareza o documento de especificação de requisitos"*.

FIGURA 13 - Fluxo de atividades da prática “Manter e descrever com clareza o documento de especificação de requisitos”



Fonte: Elaborado pelo autor (2024)

2.5.3.9 Espaço Atividades

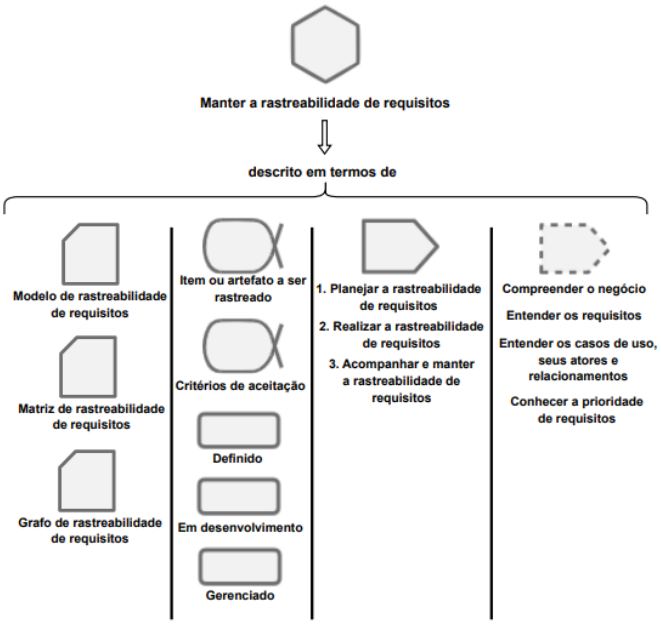
Compreender o negócio: Necessário compreender o negócio que o escopo do projeto está inserido, considerando:

- Os requisitos de negócio: Objetivos que o projeto de software candidato deve alcançar.
- As partes interessadas, ou seja, os stakeholders.
- O escopo para o desenvolvimento.

2.5.4 Prática: Manter a rastreabilidade dos requisitos

Para apresentar o detalhamento da prática *“manter a rastreabilidade dos requisitos”*, é apresentada a estrutura, organização e informações dos seus elementos, conforme Figura 14.

FIGURA 14 - Estrutura da Prática "Manter Rastreabilidade dos requisitos"



Fonte: Elaborado pelo autor (2024)

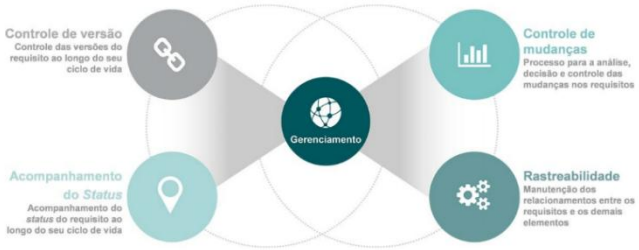
2.5.4.1 Produto de trabalho: Modelo de Rastreabilidade de requisitos

- **<u>Apresentação:</u>**

As atividades envolvidas no gerenciamento dos requisitos são (Wiegers; Beatty, 2013 *apud* Reinehr, 2018) e podem ser visualizadas na Figura 15:

- Controle de versão: os requisitos precisam ter as suas versões gerenciadas e recomenda-se a definição de uma estratégia para identificação das versões e seu acompanhamento durante todo o ciclo de vida dos requisitos e do conjunto de requisitos.
- Controle de mudanças: as mudanças nos requisitos precisam ser gerenciadas, a fim de garantir a integridade dos itens envolvidos nas mudanças.
- Acompanhamento do status: acompanhamento dos status dos requisitos, o que inclui a definição e registro dos possíveis status.
- **Rastreabilidade: envolve a possibilidade de rastrear os requisitos entre si e entre os demais elementos do sistema.**

FIGURA 15 - Categorias das atividades do gerenciamento de requisitos



Fonte: Reinehr, 2018

A rastreabilidade refere-se à possibilidade de identificar a fonte e as consequências de requisitos. Essa atividade é fundamental em situações de mudanças nos requisitos ou devido ao surgimento de falhas durante o desenvolvimento, pois a partir da rastreabilidade é possível localizar a origem desses problemas e o impacto dessas mudanças.

Independentemente do tipo e motivo da mudança, precisa ser gerenciada. Como recomendação Wiegers e Beatty (2013) *apud* Reinehr (2018) de uma efetiva gestão das mudanças, a organização deve garantir que:

- As mudanças de requisitos solicitadas e propostas, devem ser avaliadas antes de serem comprometidas.
- Os indivíduos adequados tomem decisões de negócio embasadas sobre as mudanças solicitadas.
- A atividade de mudança deve ser transparente para as partes interessadas afetadas.
- As alterações aprovadas sejam informadas a todos os participantes impactados.
- O projeto incorpore mudanças de requisitos de forma consistente e eficaz.
- Se uma mudança de requisitos for realizada sem os devidos cuidados, afetará tanto a qualidade quanto a satisfação do usuário podem ser comprometidas.

A rastreabilidade acomoda os seguintes benefícios:

- **Identificar requisitos faltantes:** buscar requisitos de negócios que não são rastreados para os requisitos de usuário e que não são rastreados para nenhum requisito funcional, o que permite descobrir se existem pontos que não foram considerados ao longo do desdobramento dos requisitos.

- **Encontrar requisitos desnecessários:** procurar por quaisquer requisitos funcionais que podem não ser necessários.
- **Realizar a certificação e conformidade:** possibilidade de usar informações de rastreabilidade quando for certificar um produto crítico para segurança e demonstrar que todos os requisitos foram implementados, embora isso não confirme que eles foram implementados corretamente.
- **Realizar a análise de impacto de mudanças:** sem informações de rastreamento, há uma boa chance de ignorar um elemento do sistema que seria afetado se fosse incluído, apagado ou modificado um requisito em particular.
- **Manter o software:** informações de rastreamento confiáveis facilitam a sua capacidade de fazer mudanças de forma correta e completa durante a manutenção. Quando as políticas corporativas ou regulamentações governamentais mudam, os sistemas de software muitas vezes precisam ser atualizados. Uma tabela que mostra onde cada regra de negócio aplicável foi abordada nos requisitos funcionais, projetos e código facilita a realização das mudanças necessárias de forma adequada.
- **Acompanhar o projeto:** o registro preciso do status de implementação da funcionalidade planejada quando tiver rastreabilidade. A ausência de links indica produtos de trabalho que ainda não foram criados.
- **Realizar a reengenharia:** é possível listar as funções em um sistema existente que está sendo substituido e rastreá-las até onde são endereçadas aos requisitos do novo sistema e componentes de software.
- **Promover a reutilização:** as informações de rastreabilidade de reuso facilitam a reutilização dos componentes do produto, identificando pacotes de requisitos, projetos (design), códigos e testes relacionados.
- **Executar testes:** quando um teste falha, as ligações entre os testes, os requisitos e o código apontam os desenvolvedores para as áreas prováveis a serem examinadas para o defeito. A rastreabilidade apoia, portanto, as correções de defeitos encontrados em testes.

A rastreabilidade dos requisitos vai garantir que (CMMI, 2018):

- Os requisitos foram totalmente atendidos nos entregáveis para o cliente.

- Os requisitos de baixo nível podem ser rastreados a uma fonte válida (solicitante).
- Os requisitos foram totalmente implementados.
- Os requisitos foram verificados e validados.
- Os relacionamentos com outras entidades, como produtos de trabalho intermediários e finais, podem:
    - Incluir dependências nas quais a mudança em um requisito pode implicar na mudança em outros requisitos.
    - Apoiar-se no projeto (design) e na avaliação do impacto das mudanças.
    - Afetar a avaliação do impacto das mudanças.
    - Apoiar a mudança antecipada do escopo.
- O projeto (design) e outras documentações refletem diretamente nos requisitos.
- Os planos de teste e casos de teste refletem os requisitos.

- **Explicação**

A implementação da prática de rastreabilidade oferece benefícios ao desenvolvimento de software sobre as informações rastreáveis (Pohl; Rupp, 2015):

- **Verificabilidade:** A rastreabilidade de requisitos permite verificar se determinado requisito foi implementado no sistema.
- **Identificação de propriedades desnecessárias do sistema:** A rastreabilidade de requisitos permite identificar as propriedades desnecessárias. Para isso, cada propriedade do sistema (funcional ou qualitativa) é inspecionada para determinar se tal propriedade contribui para a implementação de um determinado requisito no sistema.
- **Identificação dos requisitos desnecessários:** Rastrear requisitos a partir da sua origem facilita identificar requisitos que não contribuem com o objetivo do sistema. Sendo assim, não há necessidade de implementar tais requisitos.
- **Análise de impacto:** A rastreabilidade de requisitos permite analisar os efeitos das mudanças nos requisitos, por exemplo: sinalizar os artefatos que envolvem requisitos que devem ser modificados quando os requisitos que lhe apoiam passam por alguma mudança.

- **Reusabilidade:** A rastreabilidade de requisitos permite a reutilização de artefatos de requisitos em outros projetos.
- **Determinação de responsabilidade:** A rastreabilidade de requisitos permite a atribuição retroativa de esforços de desenvolvimento para um requisito.
- **Manutenção:** A rastreabilidade sintetiza a manutenção do sistema, por exemplo: a causa e o efeito de falhas podem ser identificadas, os componentes do sistema que são impactados pela falha podem ser determinados e o esforço para corrigir o erro por trás da falha pode ser estimado.

- **<u>Níveis de detalhes</u>**
  - **Rastreabilidade de requisitos deve ser definida por finalidade:**

Devido a limitação de recursos das organizações direcionadas ao desenvolvimento de projetos de software, se torna inviável capturar todas as informações concebíveis que possibilitam a rastreabilidade dos requisitos ao longo do ciclo de desenvolvimento do sistema (Pohl; Rupp, 2015).

Para implementar a rastreabilidade de requisitos de maneira efetiva, as informações a serem registradas devem ser definidas conforme a sua **finalidade**.

Definir a rastreabilidade sem considerar uma finalidade clara, terá um impacto no resultado dessa atividade, pois as informações registradas podem não ser proveitosas no projeto de desenvolvimento (Pohl; Rupp, 2015).

  - **Classificar o relacionamento de Rastreabilidade de requisitos**

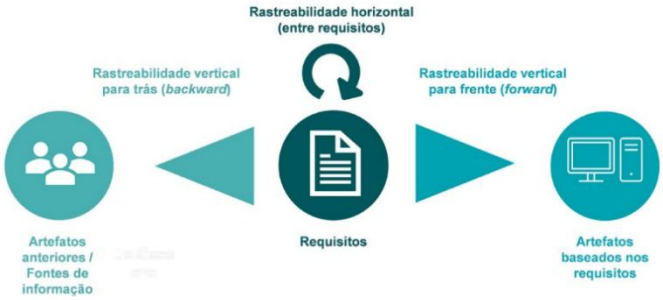
A rastreabilidade de requisitos propõe as seguintes categorias e deve ser definida junto à finalidade (Pohl; Rupp, 2015; Reinehr, 2018; Vazquez; Simões, 2016):

- **Rastreabilidade pré-especificação de requisitos ou rastreabilidade para trás (backward traceability):** Consiste no relacionamento de rastreabilidade entre requisitos os artefatos que apoiam a atividade de requisitos, como exemplo, artefatos como a fonte ou a origem de um requisito (artefatos prévios), apresentado nas Figuras 16 e 17.
- **Rastreabilidade pós-especificação de requisitos ou rastreabilidade para frente (forward traceability):** Envolve as informações de rastreabilidade entre requisitos e artefatos resultantes de atividades subsequentes de desenvolvimento. Como exemplos de artefatos, podem ser componentes,

implementação ou casos de testes, associados a um requisito e podem ser visualizadas nas Figuras 16 e 17.

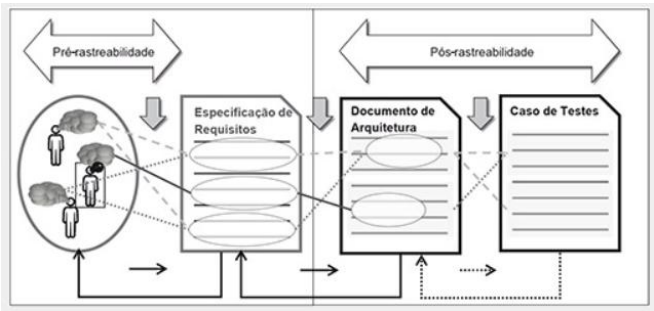
- **Rastreabilidade entre requisitos, também chamada de rastreabilidade vertical:** Considera o mapeamento de dependências entre requisitos. Como exemplo, destaca-se a rastreabilidade voltada à informação que um requisito refina, generaliza ou substitui outro requisito.

FIGURA 16 - Exemplo 1 de pré e pós-rastreabilidade dos requisitos



Fonte: Reinehr, 2018

FIGURA 17 - Exemplo 2 de pré e pós-rastreabilidade dos requisitos

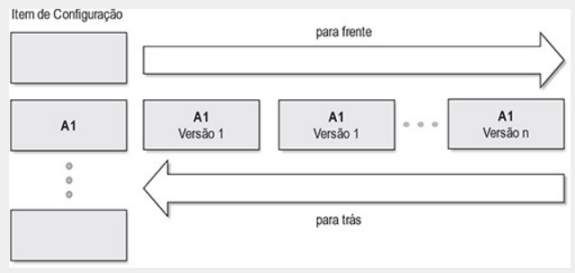


Fonte: Vazquez; Simões, 2016

No contexto da classificação do relacionamento de rastreabilidade, é possível vinculá-los à rastreabilidade horizontal ou vertical.

- **Rastreabilidade Horizontal:** Recomenda-se quando deseja-se estabelecer vínculos entre as versões diferentes de requisitos ou artefatos em determinada fase do ciclo de vida do projeto, ou, ainda, quando pretende-se vincular requisitos e artefatos que estão em um mesmo nível (Vazquez; Simões, 2016), conforme Figura 18.

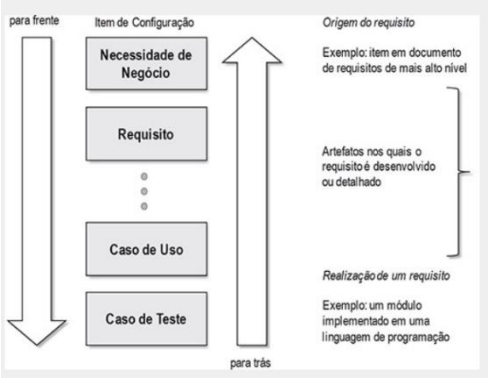
FIGURA 18 - Exemplo de rastreabilidade horizontal entre versões do mesmo artefato



Fonte: Vazquez; Simões, 2016

- **Rastreabilidade Vertical:** Sugere-se a rastreabilidade vertical quando pretende-se estabelecer vínculos entre requisitos ou artefatos que estão em diferentes fases do ciclo de vida do projeto (Vazquez; Simões, 2016), demonstrada na Figura 19.

FIGURA 19 - Exemplo de rastreabilidade vertical entre requisitos e artefatos produzidos em diferentes fases do projeto



Fonte: Vazquez & Simões, 2016

- **Definir a técnica para rastreabilidade de requisitos**

As informações presentes na rastreabilidade de requisitos podem ser representadas por referências textuais, hyperlinks, matriz de rastreabilidade e grafos de rastreabilidade.

Após definir o tipo de rastreabilidade e sua finalidade, deve-se definir a técnica que será usada para implementar a rastreabilidade.

- **Referências textuais e hyperlinks:** Consiste em sinalizar o artefato-alvo como uma referência textual no requisito, registrada no artefato inicial ou vincular hyperlinks entre o artefato inicial e o artefato-alvo. Conectando artefatos, diferentes hyperlinks com conexões semânticas específicas podem ser consideradas (Pohl; Rupp, 2015).

- **Matriz de rastreabilidade:** Uma matriz de rastreabilidade visa a representação da relação entre os requisitos e outros artefatos da engenharia de software, sendo composta por linhas e colunas (Pressman; Maxim, 2021).
  - **Grafos de rastreabilidade:** Grafo no qual os seus nós representam artefatos e as linhas, os relacionamentos entre os artefatos (Pohl; Rupp, 2015).

- **Registrar as informações da rastreabilidade dos requisitos**

Após definir **os tipos de rastreabilidade e sua finalidade**, devem ser definidos os itens que serão rastreados e a identificação desses itens é de responsabilidade do time de desenvolvimento, gestores e demais stakeholders.

Devem ser informados:

- **Projeto de software: Identificação do projeto**
- **Responsáveis: Responsáveis pela rastreabilidade de rastreabilidade**
- **Data: Data do registro da rastreabilidade**
- **Os itens a serem rastreados como:**
  - Fontes de informação (origem do requisito)
  - Requisitos críticos para o negócio
  - Requisitos que mais impactam na arquitetura
  - Requisitos mais sujeitos a mudanças (maior volatilidade)
  - Requisitos mais críticos sob a ótica do teste (mais difíceis de testar, por exemplo)
  - Códigos críticos para o sistema
  - Casos de teste críticos para o sistema
- **Relacionamento entre os artefatos e itens a serem rastreados.**

2.5.4.2 Produto de trabalho: Matriz de Rastreabilidade de requisitos

- **Apresentação**

Técnica que facilita a visualização da rastreabilidade documentada entre requisitos *versus* requisitos e requisitos *versus* outros itens. Na matriz, incluem-se os itens rastreados nos eixos de uma tabela e marcam-se os pontos de interseção (Vazquez; Simões, 2016), apresentada nas Figuras 20, 21 e 22.

Caso o projeto tenha poucos requisitos, uma tabela ou planilha normalmente é suficiente. Porém, se o projeto manter um grande volume de requisitos, recomenda-se usar uma ferramenta especializada, pois uma planilha é inviável para atender o esforço da manutenção na rastreabilidade.

A matriz serve de atividade complementar às análises de impactos das mudanças de requisitos solicitadas.

- **<u>Explicação</u>**

  Uma matriz de rastreabilidade possui linhas e colunas:

  - As linhas da matriz de rastreabilidade são rotuladas com os nomes dos requisitos.
  - As colunas podem ser rotuladas com o nome de um outro artefato da engenharia de software, como um elemento do projeto ou um caso de teste.
  - A presença de um vínculo (relacionamento) entre os dois artefatos é sinalizada na célula da matriz.
  - As matrizes de rastreabilidade podem ser usadas para garantir que os artefatos de engenharia considerem todos os requisitos.

FIGURA 20 - Exemplo 1 de matriz de rastreabilidade entre requisitos

Requisitos / Reqs. Dependentes	Req1	Req2	Req3	Req4
Req1		X	X	
Req2	X		X	
Req3	X	X		
Req4	X	X		

Fonte: Kechi, 2012

FIGURA 21 - Exemplo 2 de matriz de rastreabilidade entre requisitos

	Fase 1		Fase 2		Fase 3	
Produtos / Requisitos	Prod1	Prod2	Prod3	Prod4	Prod5	Prod6
Req1	X		X			
Req2	X	X		X	X	
Req3	X					X
Req4	X		X			X

Fonte: Kechi, 2012

FIGURA 22 - Exemplo 3 de matriz de rastreabilidade de requisitos elaborada em planilha eletrônica

	Requisitos funcionais			Requisitos não funcionais			Casos de Uso			Casos de Teste			Módulos		
	RF01	(...)	RF20	RNF1	RNF2	RNF3	UC1	(...)	UC12	CT1.1	(...)	CT20.1	M1	(...)	M54
Gerente de Vendas			X			X									
Diretor de Vendas	X														
Vendedor															
OE1	X														
OE2															
OE3			X												
RF01					X		X			X					X
(...)															
RF20						X			X			X	X		
RNF1															
RNF2															
RNF3															
UC1															
(...)															
UC12															
CT1.1															
(...)															
CT20.1															
M1															
(...)															
M54															

Fonte: Reinehr, 2018

- **<u>Níveis de detalhes</u>**
- **Planejada:**

  Os elementos da matriz de rastreabilidade de requisitos são definidos:

  - O projeto de software foi identificado.
  - A finalidade da rastreabilidade de requisitos.
    - Os responsáveis pela rastreabilidade de requisitos foram definidos.
    - A data foi informada.
    - O tipo de rastreabilidade foi definido (pré, pós-rastreabilidade ou ambos; Rastreabilidade horizontal ou vertical).
    - Os itens a serem rastreados foram definidos.

- **Em execução**

A matriz de rastreabilidade é construída considerando o tipo de rastreabilidade, os itens a serem rastreados e seus relacionamentos. **No entanto, a matriz deve ser preenchida quando todos os requisitos forem negociados e acordados com o cliente no início do ciclo de desenvolvimento de software** (Kechi, 2012).

- Na primeira coluna, são informados todos os itens que serão rastreados, os quais podem ser:
  - Os requisitos de negócios, requisitos funcionais, requisitos não funcionais, casos de uso, casos de teste ou módulos de código.

- Na primeira linha são listados os mesmos elementos ou aqueles que pretende-se rastrear o relacionamento entre eles, conforme a finalidade dessa rastreabilidade.
- Após, são sinalizados com um “X” a célula da matriz que deseja-se rastrear o relacionamento entre determinados itens.

- **Realizada**

A matriz é preenchida e acompanhada ao longo do desenvolvimento do ciclo de desenvolvimento do software. Os itens da matriz são observados e gerenciados de acordo com o relacionamento entre eles e as mudanças são analisadas, assim como o seu impacto no ciclo de desenvolvimento de software.

2.5.4.3 Produto de trabalho: Grafo de Rastreabilidade

- **Apresentação**

Representado por um gráfico no qual os nós representam artefatos (itens a serem rastreados) e as linhas o relacionamento entre eles, evidenciado na Figura 23.

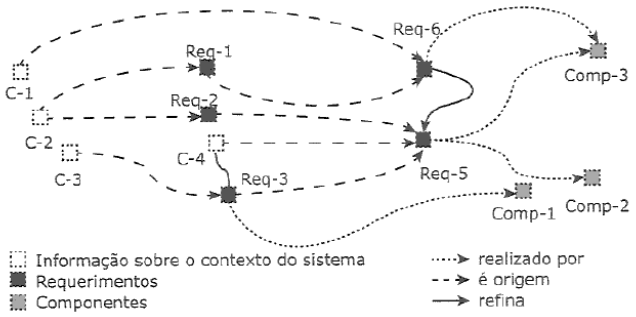
A diferença entre artefatos e os tipos de rastreabilidade pode ser definida por meio da designação de diferentes atributos para os nós e linhas do grafo.

- **Explicação**

No grafo, um tipo de nó deve ser definido para cada tipo de artefato de desenvolvimento. A letra “C” representa informações sobre o contexto do software; “Req-n” representa o requisito; e “Comp-n”, componentes.

Também, são definidos três tipos de linhas que representam três tipos de relacionamento: **executado por**; **é origem**; **refina**.

FIGURA 23 - Exemplo de Grafo de rastreabilidade de requisitos



Fonte: Pohl & Rupp, 2015

- **<u>Níveis de detalhes</u>**
- **Planejado**

  Os elementos do grafo de rastreabilidade de requisitos são definidos:

  - O projeto de software foi identificado.
  - A finalidade da rastreabilidade de requisitos.
  - Os responsáveis pela rastreabilidade de requisitos foram definidos.
  - A data foi informada.
  - Os artefatos e sua identificação a serem rastreados foram definidos.
  - Foram definidos um tipo de nó para cada tipo de artefato, bem como as linhas que representam os três tipos de relacionamento: **executado por**; **é origem**; **refina**.
- **Em execução**

O Grafo de rastreabilidade é construído considerando os tipos de relacionamentos para cada tipo de artefato a serem rastreados. **No entanto, o grafo deve ser preenchido quando todos os requisitos forem negociados e acordados com o cliente no início do ciclo de desenvolvimento de software** (Kechi, 2012).

- **Realizado**

O grafo é preenchido e acompanhado ao longo do desenvolvimento do ciclo de desenvolvimento do software. Os artefatos do grafo são observados e gerenciados de acordo com o relacionamento entre eles e as mudanças são analisadas assim como o seu impacto no ciclo de desenvolvimento de software.

2.5.4.4 Alpha: Item ou Artefato a ser rastreado

Um item ou artefato a ser rastreado pode ser caracterizado como um requisito de negócio, requisito funcional, requisito não funcional, caso de teste, parte do código.

Os itens podem estar no mesmo nível do ciclo de desenvolvimento de software ou em níveis diferentes, dependendo da finalidade da rastreabilidade e do tipo de relacionamento entre eles.

2.5.4.5 Estado Alpha: Item ou Artefato a ser rastreado

Os estados alpha do item ou artefato a ser rastreado são:

- **Definido:** Os itens a serem rastreados foram definidos, bem como o relacionamento entre eles.
- **Em desenvolvimento:** Os itens ou artefatos estão sendo acompanhados e atualizados conforme solicitação de mudanças.
- **Gerenciado:** Os itens ou artefatos e suas dependências foram verificados, implementados e entregues.

2.5.4.6 Alpha: Relacionamento entre os itens e/ou Artefatos

Um relacionamento entre os itens e/ou artefato da rastreabilidade de requisitos é um vínculo, ou seja, um elo de ligação que caracteriza uma dependência entre eles.

2.5.4.7 Estado Alpha: Relacionamento entre os itens e/ou Artefatos

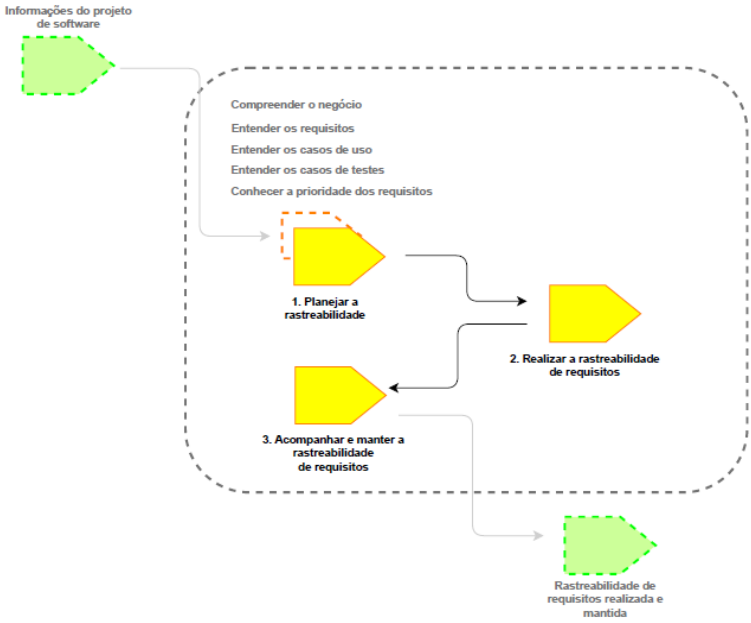
Os estados alpha do relacionamento entre os itens e/ou artefatos são:

- **Definido:** O relacionamento entre os itens ou artefatos foi definido.
- **Em desenvolvimento:** O relacionamento entre os itens ou artefatos está sendo acompanhado e atualizado conforme solicitação de mudanças. **Gerenciado:** O relacionamento dos itens ou artefatos foi verificado, implementado e entregue.

2.5.4.8 Atividades

A Figura 23 apresenta o fluxo das atividades da prática.

FIGURA 24 - Sequência das atividades da prática



Fonte: Elaborada pelo autor (2024)

As atividades são descritas como segue.

1. **Planejar a rastreabilidade de requisitos**

No planejamento da rastreabilidade de requisitos, devem ser definidas as seguintes informações fundamentais:

- Informar o projeto, data e responsáveis pela rastreabilidade de requisitos.
- Descrever a finalidade da rastreabilidade de requisitos, justificando sua necessidade.
- Informar o tipo de rastreabilidade de requisitos, conforme sua classificação.
- Informar os itens e/ou artefatos da rastreabilidade de requisitos.
- Definir o mecanismo para realizar a rastreabilidade de requisitos.

Recomendação: Pode ser utilizada uma ferramenta de gestão para incluir essas informações sobre a rastreabilidade centralizando-a e deixando-a visível para os membros do time de desenvolvimento.

2. **Realizar a rastreabilidade de requisitos**

Após planejar as informações fundamentais da rastreabilidade de requisitos, deve-se registrar e especificar o vínculo entre os itens e/ou artefatos, conforme inicialmente planejado.

Dependendo do mecanismo de representação adotado, pode-se utilizar ferramentas específicas para gerenciamento de requisitos apoiado na atividade de rastreabilidade de requisitos.

3. **Acompanhar e manter a rastreabilidade de requisitos**

A rastreabilidade de requisitos deve ser acompanhada e gerenciada durante o desenvolvimento do software, pois se mudanças forem propostas e ocorrerem, irá refletir diretamente na rastreabilidade. Dessa forma, é necessário realizar a manutenção da rastreabilidade.

2.5.4.8 Espaço Atividade

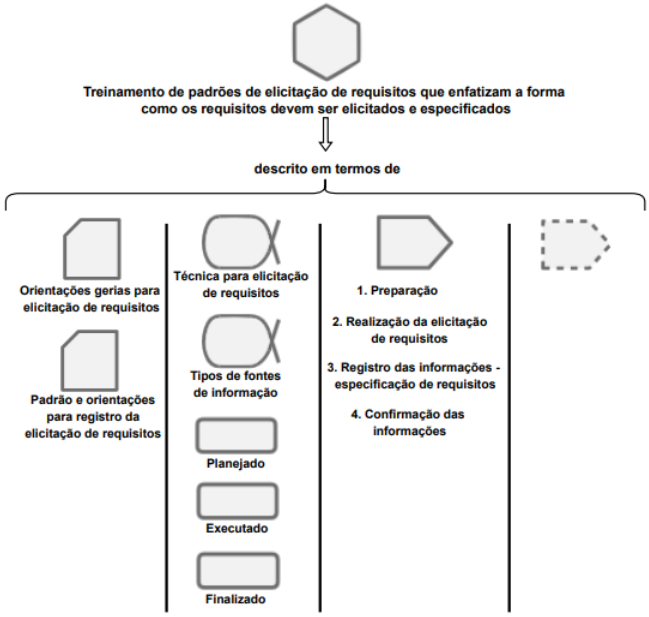
- **Compreender o negócio:** Necessário compreender o negócio que o escopo do projeto está inserido, considerando:
  - Os requisitos de negócio: Objetivos que o projeto de software candidato deve alcançar.
  - As partes interessadas, ou seja, os stakeholders.
  - O escopo para o desenvolvimento.

- **Entender e identificar os requisitos de software:** Utilizar as informações da prática de documento de requisitos.
- **Identificar, definir e compreender os casos de uso, seus atores e relacionamentos:** Aproveitar informações da prática de Caso de uso.
- **Identificar, definir e compreender os casos de testes:** Aproveitar informações da prática de Caso de uso.
- **Conhecer a prioridade dos requisitos:** Priorizar refere-se a dar um valor de importância entre os requisitos. Consiste em ampliar o valor do projeto, determinando que as coisas mais importantes sejam desenvolvidas primeiro (Vazquez; Simões, 2016). O requisito é priorizado de acordo com a respectiva importância e estabilidade, seguindo as preferências dos stakeholders.

2.5.5 Prática: Treinamento de padrões de elicitação de requisitos

Esta prática enfatiza como os requisitos devem ser elicitados. A Figura 25 apresenta os elementos que detalham a prática "*Treinamento de padrões de elicitação de requisitos*".

FIGURA 25 - Estrutura da prática "Treinamento de padrões de elicitação de requisitos"



Fonte: Elaborada pelo autor (2024)

2.5.5.1 Produto de trabalho: Orientações gerais das técnicas para elicitação de requisitos – Definir as técnicas

- **<u>Apresentação</u>**

A elicitação de requisitos, também denominada de "captura de requisitos", "descoberta de requisitos" e "aquisição de requisitos", reside na origem dos requisitos de software e como o engenheiro de software ou analista de requisitos pode coletá-los e compreendê-los (Washizaki, 2024). Muito embora não se tenha garantia de que um conjunto de requisitos esteja completo, a elicitação bem realizada ajuda a minimizar a incompletude (Washizaki, 2024).

Caracteriza-se como a primeira atividade na construção de uma compreensão do problema que o software precisa resolver. Resumindo, **é um processo de aquisição de conhecimento, onde se aplicam técnicas para compreender o negócio a ser impactado pelo projeto de software, identificar os stakeholders e para identificar e refinar os requisitos**

de software (Vazquez; Simões, 2016). Embora não haja garantia de que um conjunto de requisitos esteja completo, a elicitação bem executada ajuda a minimizar a incompletude (Washizaki, 2024).

É limitada à atividade humana, na qual as partes interessadas são identificadas e os relacionamentos são definidos entre o time de desenvolvimento e os stakeholders, sendo um dos princípios essenciais de um bom processo de elicitação de requisitos, a comunicação efetiva entre as várias partes interessadas.

Informar o escopo do projeto é considerado um elemento delicado da elicitação de requisitos, pois consiste em fornecer uma descrição do software que está sendo especificado, sua finalidade e priorizar as entregas para garantir que as necessidades comerciais mais importantes do cliente sejam satisfeitas primeiro (Washizaki, 2024).

No entanto, essas informações iniciais minimizam o risco de especialistas em requisitos dedicarem tempo elicitando requisitos de baixa importância ou aqueles pouco relevantes, quando o software é entregue. Sobretudo, a descrição deve ser escalável e extensível para acomodar outros requisitos não apontados inicialmente (Washizaki, 2024).

- **Explicação**

Os requisitos de software podem ser obtidos de diferentes fontes e é fundamental que as fontes potenciais sejam identificadas e analisadas (Washizaki, 2024).

Uma parte interessada pode ser qualquer pessoa, grupo ou organização que:

- Esteja envolvida diretamente no projeto;
- Seja afetada pelo resultado do projeto;
- Possa interferir no resultado do projeto.

Partes interessadas para projetos de software normalmente incluem (mas não estão limitadas a essas):

- Clientes: Responsáveis pela solicitação e pagamento do software que será desenvolvido. seja construído.
- Clientes: Responsáveis pela decisão de liberar o produto de software para produção.
- Usuários: Interagem direta ou indiretamente com o software e podem ser classificados em classes distintas de usuários, como: usuários que variam em frequência de uso, tarefas executadas, habilidade, nível de conhecimento, privilégio.

- Especialistas do domínio.
- Equipe de operações.
- Equipe de suporte de produto de primeiro nível.
- Órgãos profissionais relevantes.
- Agências reguladoras.
- Grupos de interesse especial.
- Desenvolvedores.

Classes de partes interessadas são agrupamentos de indivíduos que compartilham perspectivas, objetivos e necessidades semelhantes. Trabalhar em um projeto de software a partir dessas classes, em vez de considerar cada parte interessada isoladamente, tende a gerar insights adicionais e mais abrangentes sobre requisitos, expectativas e impactos do sistema.

Muitos projetos se beneficiam da realização de uma análise de partes interessadas para identificar o máximo possível de classes de partes interessadas importantes. Isso reduz a possibilidade de que os requisitos sejam tendenciosos para partes interessadas mais bem representadas e para longe de partes interessadas menos bem representadas. A análise de partes interessadas pode também informar a negociação e a resolução de conflitos quando os requisitos de uma classe de partes interessadas entram em conflito com os requisitos de outra.

Outras fontes de requisitos que não sejam partes interessadas, podem incluir:

- Documentação, como requisitos para versões anteriores, declarações de missão, conceito de operações;
- Outros sistemas;
- Contexto empresarial que podem incluir políticas e processos organizacionais;
- Ambiente de computação.

Pontos importantes na elicitação de requisitos (Washizaki, 2024):

- **Metas:** Refere-se aos objetivos gerais de alto nível do software. Os objetivos visam a motivação para o software, mas geralmente são vagamente formulados. Engenheiros de software ou analistas de requisitos precisam ter atenção à proposta de valor (relativo à prioridade) e custo dos objetivos. Realizar um estudo de viabilidade é um mecanismo de baixo custo para atender essa necessidade.
- **Conhecimento de domínio:** O engenheiro de software ou analista de requisitos precisa compreender o domínio do software. O conhecimento de domínio fornece o *background* necessário para absorver todo o conhecimento de

requisitos durante a elicitação. As associações entre conceitos relevantes dentro do domínio do aplicativo devem ser identificadas.

- **Partes interessadas (stakeholders):** O engenheiro de software ou analista de requisitos precisa identificar e gerenciar as percepções de muitos tipos diferentes de partes interessadas quando necessário, pois, às vezes, as partes interessadas podem entrar em conflito. Para minimizar esses conflitos, é importante organizar reuniões regulares com os stakeholders, oportunizando expressar suas preocupações para chegarem em um ponto comum a todos.
- **Regras de negócios:** Representam as declarações que definem ou restringem algum aspecto da estrutura ou do comportamento do próprio negócio. Por exemplo: "Um aluno não pode se registrar nos componentes curriculares do próximo semestre se houver pendências de pagamento das mensalidades". Essa regra de negócio seria uma fonte de requisitos para o software de registro de cursos de uma universidade.
- **O ambiente operacional:** Os requisitos serão derivados do ambiente em que o software será executado. Como exemplos de ambientes operacionais, destacam-se, restrições de tempo do software associado ao tempo real ou restrições de desempenho em um ambiente de negócios. Estes devem ser coletados ativamente porque podem afetar a viabilidade e o custo do software, assim como restringir as escolhas de design.
- **O ambiente organizacional:** O software geralmente é desenvolvido para dar suporte a um processo de negócios, cuja seleção pode estar condicionada a estrutura, cultura e política interna da organização. O engenheiro de software ou analista de requisitos precisa ser sensível a isso pois, um novo software não deve impor mudanças não planejadas no processo de negócios.

- **<u>Níveis de detalhes</u>**
- **Definição das técnicas de elicitação de requisitos**

Após identificarem as fontes de requisitos, o engenheiro de software ou analista de requisitos pode iniciar a elicitação de informações de requisitos, porém, os requisitos raramente são elicitados prontos e completos. O engenheiro de software ou analista de requisitos formula os requisitos a partir das informações solicitadas inicialmente.

Para realizar a elicitação de requisitos são propostas diferentes técnicas para obter informações de requisitos relevantes para defini-los corretamente. No entanto, algumas técnicas funcionam melhor com certas classes de stakeholders do que outras (Washizaki, 2024).

A elicitação, por vezes, é uma tarefa difícil, pois os stakeholders podem ter dificuldade em relatar ou descrever suas tarefas, esquecendo informações importantes ou deixar de relatá-las propositalmente. Nessa perspectiva, o engenheiro de software ou analista de requisitos deve ter a sensibilidade de identificar possíveis conflitos de interesse e articular a comunicação de forma que possibilite elicitar informações claras, relevantes e corretas (Washizaki, 2024)

Tentando tornar a elicitação de requisitos eficaz, podem ser combinadas várias técnicas de elicitação, conforme a necessidade do projeto e do contexto.

Dentre as técnicas que podem ser definidas para a elicitação, destacam-se:

- **Entrevistas:** Entrevistar as partes interessadas é um meio "tradicional" de elicitar requisitos. É importante entender as vantagens e limitações das entrevistas e como elas devem ser conduzidas (Washizaki, 2024).
- **Histórias de usuário** (Washizaki, 2024)**:** Essa técnica é comumente usada em métodos ágeis e se refere a descrições curtas e de alto nível da funcionalidade necessária expressa em termos do cliente. Uma história de usuário típica tem o formato: "**Como <função>, eu quero <objetivo/desejo> para que <benefício>**."
- **Cenários de uso:** Os cenários fornecem um meio interessante e valioso para fornecer contexto para a elicitação dos requisitos do usuário. Permitem que o engenheiro de software forneça uma estrutura para perguntas sobre tarefas do usuário, permitindo que perguntas do tipo "**e se**" e "**como isso é feito**" sejam realizadas. O tipo mais comum de cenário é a descrição do caso de uso (Washizaki, 2024; Sommerville, 2018).
- **Observação (Etnografia):** A importância do contexto do software dentro do ambiente organizacional levou à adaptação de técnicas observacionais, como a etnografia, para a obtenção de requisitos. Engenheiros de software aprendem sobre as tarefas do usuário emergindo no ambiente e observando como os usuários realizam suas tarefas, interagem entre si e com ferramentas de software e outros recursos (Sommerville, 2018).
- **Prototipagem de interface de usuário de baixa ou alta fidelidade** (Washizaki, 2024)**:** Esta técnica é uma ferramenta valiosa para esclarecer requisitos ambíguos, ausentes ou conflitantes. Oferecem aos usuários um contexto dentro do qual eles podem entender melhor quais informações precisam fornecer por meio de protótipos.

- **Reuniões, incluindo brainstorming:** O objetivo dessas reuniões é proporcionar um efeito somativo, por meio do qual um grupo de pessoas pode apresentar mais insights sobre seus requisitos de software do que trabalhando individualmente. Podem fazer um brainstorming e refinar ideias difíceis de trazer à tona usando entrevistas (Washizaki, 2024).
- **Outras técnicas:** Existe uma série de outras técnicas para dar suporte à obtenção de informações de requisitos como, por exemplo: grupos focais; desenvolvimento de aplicativo conjunto (JAD); planejamento de requisitos conjuntos (JRP); análise de protocolo; questionários e pesquisas de mercado (Washizaki, 2024).

Em Washizaki **(2024) são citadas outras técnicas e fontes de requisitos sem a presença direta dos stakeholders como:**

- Versões mais antigas do sistema;
- Banco de dados de rastreamento de defeitos de versões anteriores do sistema;
- Benchmarking competitivo.
- Decomposição (por exemplo, recursos em épicos em recursos em histórias).
- Design thinking (empathize, define, ideate, prototype, test).

- **Realização da técnica**

Após as fontes de requisitos e a definição da técnica serem identificadas, a técnica é executada e as informações sobre os requisitos são coletados e registrados.

Para registrar as informações, pode-se usar ferramentas de gestão de projetos ou ferramentas com suporte para as atividades da engenheira de requisitos.

2.5.5.2 Produto de trabalho: Padrão e orientações para as atividades da elicitação de requisitos

- **<u>Apresentação</u>**

Independente da técnica empregada na elicitação de requisitos, é importante atender atividades fundamentais para sistematizar o processo de elicitação: (i) preparação para elicitação de requisitos; (ii) execução da elicitação de requisitos e, (iii) confirmação das informações da elicitação de requisitos.

- **<u>Explicação</u>**

Definir um padrão e algumas orientações sobre as atividades que compõem a elicitação de requisitos é importante para estabelecer uma comunicação unificada, na qual todos os analistas tenham a mesma compreensão do processo de elicitação seguindo as mesmas diretrizes, independente da técnica adotada.

- **<u>Níveis de detalhes</u>**
- **Preparação para elicitação de requisitos**

A preparação para elicitação consiste em garantir que todos os recursos necessários estejam organizados e preservados para a sua realização e é fundamental para garantir um bom resultado na elicitação.

Entende-se como recursos:

- Agenda das pessoas envolvidas.
- Disponibilidade de salas.
- Planejamento de viagens, quando necessário.
- Questões logísticas de modo geral.

Também, é o momento de selecionar as técnicas que serão utilizadas durante a elicitação. A escolha das técnicas pode ser influenciada pelo: domínio do negócio, cultura empresarial, habilidades da equipe e recursos disponíveis.

- Não existe a melhor técnica, mas sim, aquela que se adapta melhor em determinada necessidade ou projeto.
- Normalmente, na elicitação se utiliza mais de uma técnica.
- Importante que o analista tenha familiaridade com várias técnicas para que seja usada a mais adequada conforme o projeto ou perfil dos stakeholders.

Para qualquer necessidade de informação deve-se manter na preparação para a elicitação: **Lista de stakeholders com papéis e responsabilidades definidas** (com quem o analista irá interagir).

- **Execução da elicitação de requisitos**

Durante a execução das atividades de elicitação, o foco é coletar informações proativamente junto aos stakeholders utilizando as técnicas de elicitação definidas na preparação.

A metodologia e características de cada técnica devem ser observadas e consideradas, pois cada técnica tem suas especificidades.

Recomendações para execução da técnica (Vazquez; Simões, 2016)**:**

- Durante a execução da técnica, evitar o desvio do escopo do projeto.
- O projeto nem sempre tem o objetivo de resolver todos os problemas relatados pelos stakeholders, pois alguns podem estar fora do escopo do projeto, por isso, é importante conscientizar os stakeholders, do objetivo do projeto.
- Manter a rastreabilidade dos requisitos dos stakeholders para os requisitos de negócios, pois é a garantia que o escopo do projeto não será desviado.
- Ficar atento para capturar os atributos dos requisitos, os quais podem ser apoiados e identificados seguindo o plano de gerenciamento de requisitos. Exemplos de atributos: origem, prioridade, autor, proprietário, estado, risco, etc.
- Na captura dos requisitos e seus atributos deve-se utilizar um mecanismo para registro dessas informações, como anotá-las em um papel, documento, ferramenta, gravar (desde que autorizado pelos stakeholders).
- Não esperar que a parte interessada relate tudo o que é necessário para atender o escopo do projeto, voluntariamente. Requisitos implícitos, como a omissão de requisitos fundamentais, ocasionados pela falta de informações por parte dos stakeholders, é um problema que ocorre durante a elicitação. O papel do analista é fundamental nesses momentos, pois deve ter sensibilidade e expertise para identificar o que a parte interessada pensa que é óbvio e não menciona. Isso evitará retrabalho à frente no projeto.
- Deve haver tempo e recurso para que o analista amplie ou melhore seu conhecimento do negócio. Para isso, sugere-se organizar as partes de informação coletadas em diferentes visões visando identificar lacunas.

Recomendações para a documentação dos requisitos da elicitação durante a execução da elicitação (Vazquez; Simões, 2016):

- As técnicas adotadas durante a execução da elicitação de requisitos, definem o tipo de documentação.

- Há abordagens em que os produtos da elicitação são produtos finais, como as histórias de usuários adotadas nos métodos ágeis.
- Abordagens em que os produtos da elicitação são requisitos intermediários, a fim de registrar decisões sobre assuntos, até então, pendentes, devem documentar por meio de atas de reuniões, relatórios, registro de áudio ou vídeo, quadro de anotações, por exemplo.
    - Nesse caso, esse tipo de documentação será utilizado para confirmar a compreensão do analista com os stakeholders sobre as informações coletadas.
- Devem ser registradas também, questões em aberto e dúvidas que surgiram durante essas cerimônias e que não foram esclarecidas até então.

- **Confirmação dos resultados da elicitação de requisitos**

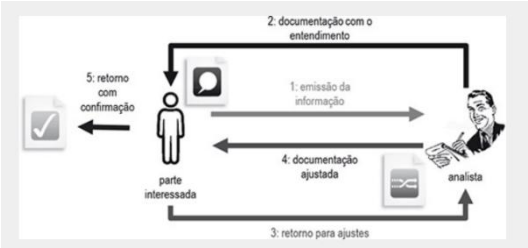
As informações durante a elicitação só podem ser confirmadas se houver um registro dessas informações, o que enaltece a documentação como mecanismo fundamental durante o processo de elicitação de requisitos. Caso não sejam documentadas, existe um grande risco de perda dessas informações.

Mesmo registradas, ainda pode ocorrer ruídos entre o relato da informação e a sua compreensão por parte do analista, justificando a necessidade da confirmação ou validação das informações junto aos stakeholders, após momento da elicitação.

Importante seguir um protocolo de comunicação para o momento de confirmação, embora deve ser algo simples, sem burocracia.

Um protocolo recomendado sobre como **receber a informação, documentar e confirmar é denominado "Protocolo do garçom"**, o qual é simples e eficaz, demonstrado na Figura 26.

FIGURA 26 - Protocolo do garçom para filtrar falhas de comunicação na elicitação de requisitos



Fonte: Vazquez & Simões, 2016

2.5.5.3 Alpha: Técnica para elicitação de requisitos

Técnica de elicitação de requisitos é considerada como uma ferramenta para auxiliar o analista de requisitos na condução da etapa de elicitação e compreensão dos requisitos do software (Reinher, 2018).

Existem várias técnicas à disposição dos analistas de requisitos. Algumas têm intervenção humana e outras são voltadas a outros tipos de fontes de informação.

Exemplos de técnicas recomendadas para a elicitação de requisitos:

Entrevista

Uma das técnicas mais populares para elicitação de requisitos a partir de fontes humanas. Consiste em uma conversa entre duas ou mais pessoas, envolvendo stakeholder e analista de requisitos, considerando um objetivo definido. A entrevista facilita o acesso às informações que, muitas vezes, estão apenas na memória das pessoas.

Durante o contato, é importante estabelecer uma conexão entre analista de requisitos e stakeholders, pois essa relação permite maior segurança para expor questões delicadas (Wiegers; Beatty, 2013 *apud* Reinher, 2018).

A entrevista, assim como qualquer outra técnica requer uma preparação por parte de quem irá entrevistar.

Recomenda-se que o entrevistador, organize uma lista de questões mais abrangentes no início e que, progressivamente, sejam mais detalhadas:

- As questões devem seguir uma sequência lógica e abordar um assunto de cada vez.
- O roteiro com as questões serve apenas para auxiliar o processo de entrevista, ou seja, não deve ser restritiva, pois é necessário dar abertura para o entrevistado expor situações, além das questionadas inicialmente, relacionadas à solução que será desenvolvida.

Sugere-se o uso de entrevista:

- Quando as pessoas possuem o conhecimento necessário sobre o negócio e estão disponíveis para serem entrevistadas.
- Para ouvir as informações subjetivas (desabafos, sentimentos, pontos de vista) que podem ser úteis para a definição de requisitos do sistema.
- Para identificar e compreender o fluxo de trabalho e de documentos.

Desafios durante a entrevista, pois pode não obter os resultados desejados, caso ocorra uma das seguintes situações

- Falta de expertise do condutor da entrevista.
- Falta de preparo do entrevistado.
- Tendência do entrevistado em dar respostas desagradáveis.
- Tendência do entrevistado em focar em palpites ou em respostas falsas.
- Resistência do entrevistado com respostas “não sei”, “não conheço”.

Orientações para a condução de uma boa entrevista:

- Conquistar a confiança do entrevistado se identificando e introduzindo claramente a finalidade da entrevista.
- Documentar a entrevista, seja de forma textual ou gravando-a (o entrevistado deve autorizar a gravação).
- Analisar as informações expostas pelo entrevistado, criticamente, após a entrevista (não corrija falhas, faça críticas ou discuta).
- Iniciar com questões mais abertas: “como é o trabalho que você desenvolve na empresa?”.
- Evitar termos técnicos e procurar se aproximar de termos familiares à área.
- Evitar perguntas com respostas que possibilitam apenas “sim” ou “não”, pois limitam a participação do entrevistado.
- Caso a entrevista se desvie do foco, reconduza, com habilidade.
- Se o entrevistado se recusar a informar dados, obtenha o auxílio de outros colegas em papel semelhante ou, em último caso, contacte o superior hierárquico do entrevistado.
- Esclarecer as possíveis preocupações manifestadas pelo entrevistado.
- Permitir espaço para novos contatos, da sua parte e da parte do entrevistado.
- Evitar interrupções nos relatos do entrevistado por parte do entrevistador.
- Validar as informações obtidas com relação ao objetivo definido.
- Permitir que a entrevista seja conduzida de forma online, por meio de ferramentas de comunicação, caso o entrevistado tenha limitação de disponibilidade, embora, quando possível, realize-a presencialmente.
- Garantir que a tecnologia (ferramenta, rede, internet) funcione adequadamente, evitando as interrupções, que prejudicam a condução da conversa.

Histórias de usuário ou user stories

- **Uma user story deve conter apenas informações suficientes para que os desenvolvedores possam produzir uma estimativa razoável do esforço para implementá-la.**
- O objetivo é evitar parte do desperdício que geralmente acontece em projetos onde requisitos detalhados são coletados cedo, mas se tornam inválidos antes do trabalho começar.
- Antes que uma ***user story*** seja implementada, um procedimento de aceitação apropriado deve ser escrito pelo cliente para determinar se os objetivos da história de usuário foram cumpridos.

Recomendação:

Uma boa *user story* deve ser objetiva, clara e direta, descrita ou relatada por um stakeholder que precisa obter algum retorno de valor.

Escrever a *user story* onde:

- Alguém: Stakeholders
- Quer o que: requisito
- Para obter qual retorno: valor de negócio

Questione: "como esse requisito será validado"?

- Descreva a resposta como critério de aceitação da *user story*.

Exemplo de user stories de um projeto de construção de um smartphone cuja bateria possui durabilidade superior a uma semana.

Cenários

Consiste em uma descrição de como o sistema pode ser utilizado em alguma tarefa particular. São descrições de exemplos de sessões de interação com o usuário.

Os cenários são estruturados com informações específicas mais detalhadas, coletadas, como entradas e saídas.

Recomendação (Sommerville 2018)**:**

- Um cenário inicia com uma descrição da interação.
- Durante o processo de elicitação, são ampliados detalhes para representar uma descrição completa dessa interação.

Um cenário pode incluir (Sommerville 2018)**:**

- Uma descrição do que o sistema e os usuários esperam quando o cenário se inicia.
- Uma descrição do fluxo normal dos eventos no cenário.
- Uma descrição do que pode dar errado e como esses problemas podem ser enfrentados.
- Informações sobre outras atividades que poderiam acontecer ao mesmo tempo.
- Uma descrição do estado do sistema quando finaliza o cenário.

Exemplo de cenário: Quando um aluno envia fotos para o KisTakePics, apresentado na Figura 27.

FIGURA 27 - Exemplo de user stories

Enviar fotos para o KidsTakePics

Pressuposto inicial: Um usuário ou grupo de usuários tem uma ou mais fotografias digitais para serem enviadas para o site de compartilhamento de imagens. Essas fotos estão salvas em um tablet ou notebook. Eles fizeram o *login* no site KidsTakePics.

Normal: O usuário opta por enviar as fotos e é solicitado a ele que selecione as fotos no computador a serem enviadas e escolha o nome do projeto sob o qual as fotos serão armazenadas. Os usuários também devem ter a opção de digitar palavras-chave que deverão ser associadas a cada foto enviada. Essas fotos recebem um nome criado pela conjunção do nome do usuário com o nome do arquivo da foto no computador local.

No final do envio, o sistema manda automaticamente um e-mail para o moderador do projeto, pedindo-lhe que verifique o novo conteúdo, e gera uma mensagem na tela para o usuário dizendo que essa verificação foi feita.

O que pode dar errado: Nenhum moderador está associado ao projeto selecionado. Um e-mail é gerado automaticamente para o administrador da escola pedindo para nomear um moderador do projeto. Os usuários devem ser informados de um possível atraso no procedimento para tornar suas fotos visíveis.

Fotos com o mesmo nome já foram enviadas pelo mesmo usuário. O usuário deve ser questionado se deseja enviar novamente as fotos, renomeá-las ou cancelar seu envio. Se os usuários escolherem reenviar, os originais serão sobrescritos. Se optarem por renomear, um novo nome será gerado automaticamente acrescentando um número ao nome de arquivo existente.

Outras atividades: O moderador pode estar logado no sistema e aprovar as fotos à medida que forem enviadas.

Estado do sistema ao terminar: O usuário está logado. As fotos escolhidas foram enviadas e receberam o status de 'aguardando moderação'. As fotos estarão visíveis para o moderador e para o usuário que as enviou.

Fonte: Sommerville, 2018

Observação

Técnica na qual o analista de requisitos permanece *in loco* para observar como ocorre o trabalho realizado pelos stakeholders.

Consiste, não apenas em ver e ouvir, mas em examinar fatos ou fenômenos que se deseja compreender. A observação pode ser usada para confirmar informações obtidas em outras técnicas comparando-as com a realidade.

É possível acessar outras fontes de informações, como manuais de procedimentos e obter o fluxo de documentos ao longo das tarefas.

A observação auxilia na percepção de ideias sobre pontos nos quais o projeto de software pode reduzir gargalos e reduzir trabalhos realizados de forma manual.

Sugere-se o uso da observação quando:

- O fluxo de papéis é relevante.
- Problemas de desempenho impactam à forma de executar o trabalho.
- A influência do ambiente real é necessária.

Desafios da observação

- Um dos desafios e limitação da observação é a interferência no ambiente de trabalho, mesmo sem ser intencional.
- A presença de um observador, alguém externo à empresa, pode provocar perturbações nas pessoas que pertencem aquele ambiente de trabalho, o que pode alterar as condições reais em que o trabalho é realizado.
- O observador tem seu próprio ponto de vista, o que pode gerar impressões favoráveis ou desfavoráveis.
- A observação consome um tempo significativo e, por conta disso, não será viável para ser utilizada em cada tarefa que o usuário realiza.
- O tempo de duração de uma observação pode variar, de acordo com o que se pretende observar.
- É importante certificar-se que as tarefas ou fluxo que pretende observar, ocorrerá durante o período da observação.
- O observador deve registrar a sua percepção de forma discreta para não atrapalhar o andamento das atividades laborais dos colaboradores da empresa.

Orientações durante a observação:

- Como ocorre o fluxo de atividades entre os participantes do processo?
- Existem momentos de pico de atividades?
- Existem problemas causados pela tecnologia ou pela ausência dela?
- O que acontece quando a tecnologia falha (cai a rede, por exemplo)?

- Quais são os fluxos alternativos e casos que requerem um tratamento especial?

Brainstorming (Reinher 2018)

O brainstorming visa uma *"tempestade de ideias"*, sem críticas ou análises, que ocorre em um ambiente informal, descontraído e propício à criatividade e é uma técnica que envolve um grupo de pessoas participantes.

Sugere-se o uso desta técnica quando:

- O ambiente da empresa é informal e disponível para o desenvolvimento de atividades criativas.
- Quando pretende-se encontrar soluções inovadoras e criativas.
- Quando pretende-se lançar novos produtos no mercado.

Para realizar a técnica de brainstorming, recomenda-se:

- Providenciar o material com antecedência e pode ser composto por post-it (notinhas adesivas coloridas), canetas para as anotações.
- O material deve ser suficiente para todos os participantes terem acesso considerando os objetivos da técnica.
- A base do *brainstorming* é a geração de muitas ideias, e cada uma escrita em uma notinha.
- A técnica de *brainstorming* também pode ser realizada por meio de ferramentas de apoio.

Fases do Brainstorming

A seção de brainstorming é composta por três fases:

- **Aquecimento:** Dedicada à dinâmica selecionada, na qual o problema a ser resolvido ou a temática é apresentada.
- **Ideação:** A ideação consiste na geração de ideias manifestadas pelos participantes e anotadas nos post-it, visíveis a todos e sem julgamentos. Nesta fase um conjunto expressivo de ideias é registrado pelos participantes. A fase de ideação finaliza quando o conjunto de ideias for suficiente para dar continuidade ao processo.

- **Encerramento:** Após a ideação, as ideias mais relevantes pelo grupo são selecionadas e agrupadas e, após, é dado continuidade ao processo de desenvolvimento adotado pela empresa.
    - Opcionalmente, pode ser realizada, na última etapa, uma breve análise ou votação das melhores ideias.

Orientações:

Algumas questões devem ser observadas para que o brainstorming seja realizado com sucesso:

- O brainstorming não se aplica a grupos fechados e formais.
- A técnica requer um clima descontraído sem críticas e julgamentos, pois isso inibe o fluxo de ideias.
- Quando o grupo for muito grande, o ideal é fragmentá-lo em subgrupos com, no máximo, 8 pessoas cada grupo, dependendo do contexto e dos perfis.

Prototipagem

Protótipos são representações visuais que se assemelham às informações que constarão no layout do software a ser desenvolvido.

Existem técnicas de prototipagem, desde modelos de papel, de designs de tela, a versões beta de produtos de software, sendo um forte mecanismo para elicitação de requisitos e para validação de requisitos.

Protótipos de baixa fidelidade são frequentemente preferidos para evitar a “ancoragem” dos stakeholders em características menores de um protótipo de qualidade superior que pode limitar a flexibilidade do design de maneiras não intencionais.

2.5.5.4 Estado Alpha: Técnica para elicitação de requisitos

Os estados alpha da técnica para elicitação de requisitos são:

- **Planejado:** A técnica de elicitação de requisitos foi definida, conforme a necessidade do projeto e da empresa, bem como as informações sobre analista de requisitos, stakeholders e demais fontes de informações, bem como objetivos da técnica e cronograma.

- **Executado:** A técnica de elicitação foi realizada conforme o seu planejamento, as informações foram coletadas e, após, documentadas seguindo o padrão estabelecido pela empresa.
- **Finalizado:** As informações foram confirmadas com os stakeholders, ajustadas e atualizadas quando necessárias. Após, é dado sequência ao desenvolvimento do projeto de software.

2.5.5.5 Alpha: Tipos de Informação

É importante que a origem dos requisitos esteja clara e que os potenciais stakeholders sejam envolvidos logo no início do processo de elicitação de requisitos.

Pohl e Rupp (2015) definem três tipos de fontes de informação:

- **Stakeholders:** são pessoas ou organizações que interferem (direta ou indiretamente) nos requisitos do sistema. Como exemplos, destacam-se os usuários, operadores do sistema, desenvolvedores, arquitetos, clientes, testadores.
- **Documentos:** mantém informações que podem conter requisitos, como, por exemplo, documentos legais ou regulatórios, normas, assim como documentos específicos do domínio de negócio ou da própria empresa, como documentos de requisitos ou documentos de erros em sistemas legados.
- **Sistemas em operação:** sistemas legados ou antecessores.

No método ágil SCRUM, utiliza-se o papel do *product owner (PO)* ou seja, o dono do produto, que é responsável por gerenciar o *product backlog*. As principais atribuições do PO concentram-se no gerenciamento e prioridades do *product backlog* e garantir a compreensão adequada de cada item do backlog por parte da equipe de desenvolvimento (Reinher, 2018).

Envolvimento dos stakeholders (Leffin-Gwell, 2011 *apud* Reinher, 2018):

- **Stakeholders que devem ser mantidos informados:** aqueles que precisam acompanhar o status do projeto e precisam ser mantidos atualizados e informados sobre as decisões, embora nem sempre tomem parte dessas decisões.
- **Stakeholders que devem ser consultados:** aqueles que devem ser envolvidos devido às áreas de expertise, como ocorre com arquitetos, projetistas de interface, marketing e

especialistas do domínio. Devem ser incluídos nas decisões contribuindo com a sua área de expertise.

- **Stakeholders que serão parceiros no desenvolvimento:** aqueles que devem ser envolvidos por serem partes do processo. Por exemplo, os *product owners* de outros projetos, outras equipes de desenvolvimento, analistas de negócios ou requisitos e provedores de soluções que interagem com o sistema.
- **Stakeholders que controlam os resultados:** aqueles que tomam as decisões sobre a solução desejada. Por exemplo, os executivos, gerentes de liberação, usuários-chave que vão utilizar a solução e *business owners*.

Orientações para definir as fontes de informação

Para identificar as fontes de informação dos requisitos de um projeto, recomenda-se consultar o responsável pela aquisição, que poderá indicar as áreas de negócio que terão interface com o software em desenvolvimento e apontar eventuais leis ou regulamentações que precisam ser atendidas.

Recomendações de questões para a identificação dos stakeholders (Leffin-Gwell, 2011 *apud* Reinher, 2018), apresentadas na Figura 28:

FIGURA 28 - Questões para identificação dos stakeholders

***Stakeholders* externos**	***Stakeholders* internos**
■ Quem utilizará diretamente o sistema? ■ Quem utilizará os resultados daqueles que usam o sistema? ■ Quem será responsável por dar suporte ao sistema? ■ Com quais outros sistemas o nosso sistema irá interagir? ■ Que interfaces devem ser fornecidas? ■ Quem pode fornecer orientações sobre as funcionalidades e os itens de qualidade do sistema (usabilidade, confiabilidade, desempenho e manutenibilidade)?	■ Quem precisa ser consultado sobre o escopo do projeto? ■ Quem contribuiu para o orçamento e o cronograma? ■ Quem gerencia o relacionamento de negócios entre as equipes e o cliente? ■ Quem irá determinar como e quando o sistema deve ser entregue para os clientes? ■ Quem pode afetar ou apoiar politicamente o projeto? ■ Que parceiros dependem do sistema? ■ Quem se preocupa com o processo que será usado no desenvolvimento do sistema?

Fonte: Reinher, 2018

2.5.5.6 Estado Alpha: Tipos de Informação

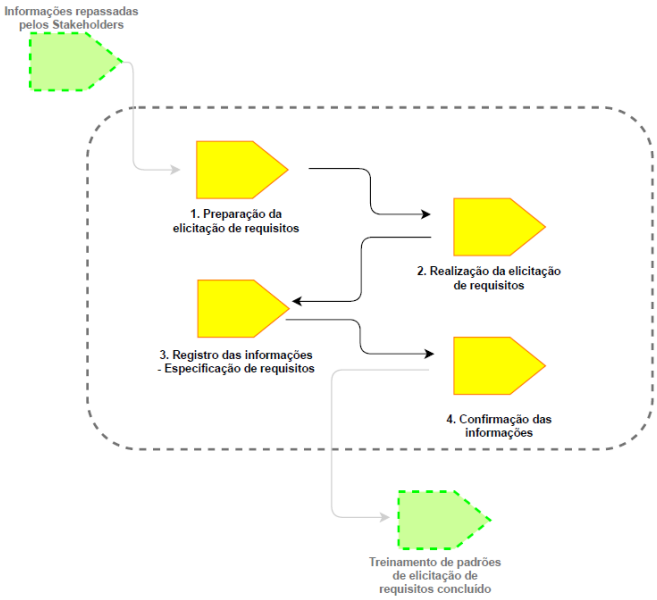
Os estados alpha dos tipos de informação são:

- **Planejado:** A fonte de informação foi definida conforme a necessidade do projeto e da empresa.
- **Executado:** A fonte de informação definida é responsável pela realização da técnica de elicitação de requisitos e pelo registro das informações coletadas.
- **Finalizado:** A fonte de informação disponibiliza o registro das informações elicitadas.

2.5.5.7 Atividades

A Figura 29 apresenta a sequência de atividades da prática.

FIGURA 29 - Sequência das atividades da prática



Fonte: Elaborado pelo autor (2024)

A descrição das atividades é apresentada a seguir:

1. **Preparação**
    - Definição das informações iniciais do projeto.
    - Definição das fontes de informação.
    - Definição da técnica de elicitação de requisitos.
2. **Execução**
    - Realizar a elicitação de requisitos por meio da técnica selecionada com as fontes de informações definidas.

3. **Registro das informações**
    - As informações coletadas durante a elicitação de requisitos devem ser documentadas, seguindo a orientação definida pela empresa e disponibilizadas para as partes interessadas.
4. **Confirmação das informações**
    - As informações documentadas devem ser apresentadas para os stakeholders, ajustadas, atualizadas e documentadas novamente.

2.6 Formalização e Implementação

A fase de Formalização é responsável pela transformação do modelo conceitual para um modelo formal, que pode ser compreendido por máquinas, e é apoiada por um editor para construção de ontologia, implementando-a (Fernández-López; Gómez-Pérez; Corcho, 2004). Optou-se pelo editor denominado Protégé[3] para implementar a OntoReD.

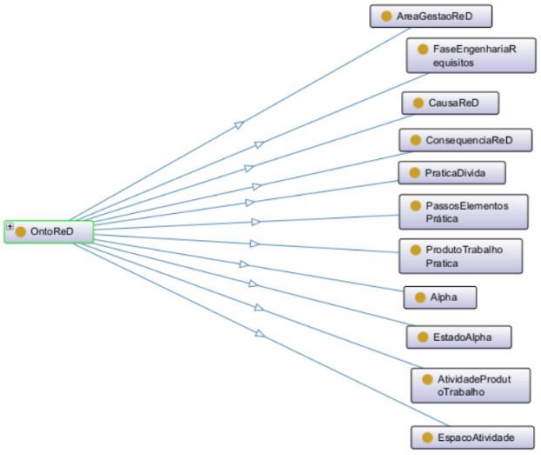
Por convenção, recomenda-se letra maiúscula para denominar o nome das classes e, quando o conceito possui mais de uma palavra para denominar a classe deve-se concatená-la, iniciando cada palavra com a letra maiúscula (Horridge; Brandt, 2011). Por exemplo, o conceito "causa da dívida de requisitos", está representado na OntoReD como: "CausaReD". Para representar as relações associativas da Ontologia, utiliza-se propriedades por meio de dois componentes: *Object Properties* que determina o relacionamento entre dois conceitos e *Data Properties* que caracteriza as propriedades dos dados, ou seja, atributos das instâncias das classes. Ambas as propriedades recomendam-se iniciar sempre por letra minúscula (Horridge et al., 2009; Miroir, 2018).

[3] https://protege.stanford.edu/

2.6.1 Construção das classes

A hierarquia das classes da OntoReD pode ser visualizada na Figura 30.

FIGURA 30 - Hierarquia das classes da OntoReD



Fonte: Capturada do Protégé – OntoReD (2023)

2.6.2 Definição dos Relacionamentos da OntoReD – Object Properties

O relacionamento entre as classes representa a dependência entre conceitos de domínio e permite definir axiomas e restrições dessas classes à ontologia. Axiomas são afirmações que declaram a verdade sobre um determinado domínio (Fernández-López, Gómez-Pérez; Juristo, 1997).

Na implementação das propriedades dos objetos na ferramenta Protégé, vinculou-se às propriedades as seguintes características: Funcional, Assimétrica, Reflexiva, Transitiva, as quais podem ser compreendidas conforme sua descrição, apresentadas no Quadro 3 e segue como referência as orientações de Horridge; Brandt (2011). A formalização da lista de *objects properties*, pode ser visualizada na Figura 31 seguindo o modelo e recomendação de Horridge; Brandt (2011).

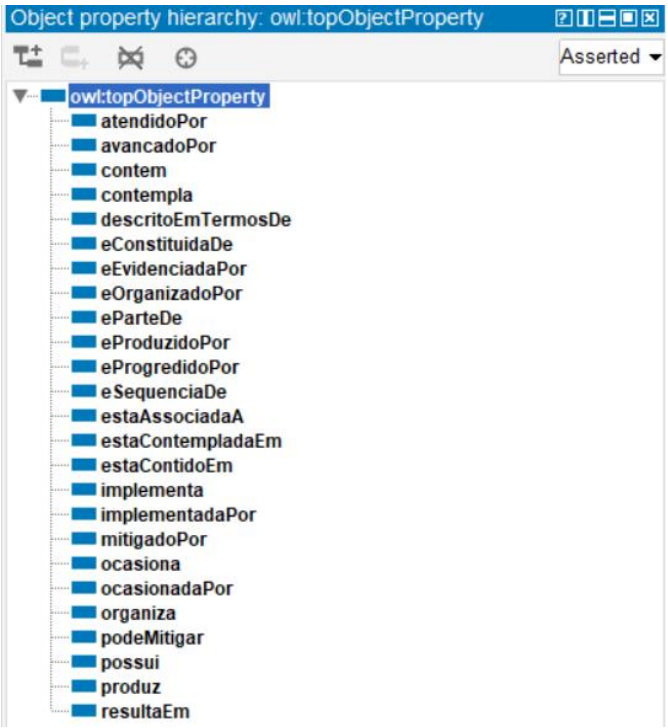
A Figura 32 demonstra exemplos de associação das classes, estabelecidas por meio das propriedades de objetos (*objects properties*) e conectam indivíduos de um domínio (*domain*) a indivíduos de um escopo (*range*). A linha “azul” demonstra as subclasses da OntoReD, e cada “linha pontilhada com uma seta” da associação, representa uma propriedade, a qual é diferenciada por cores. A relação das *objects properties* dos conceitos podem ser visualizadas no Quadro 4.

QUADRO 3 - Características das *Objects Properties*

Característica - Object Property	Descrição
Functional	Se uma propriedade é definida como funcional para um determinado indivíduo “A”, então, pode existir, até, no máximo, um indivíduo “B” relacionado ao indivíduo “A” via essa propriedade.
Transitive	Uma propriedade P é transitiva quando a propriedade relaciona o indivíduo "A" ao indivíduo "B" e, também um indivíduo "B" ao indivíduo "C", então, infere-se que o indivíduo "A" está relacionado ao indivíduo "C", por meio da propriedade P.
Asymmetric	A propriedade P é definida como assimétrica e relaciona o indivíduo “A” ao indivíduo “B”, porém, o indivíduo “B” não pode ser relacionado ao indivíduo “A” por meio de uma propriedade individual P.
Reflexive	Uma propriedade P é caracterizada como reflexiva quando ela relaciona o indivíduo “A” a ele mesmo.

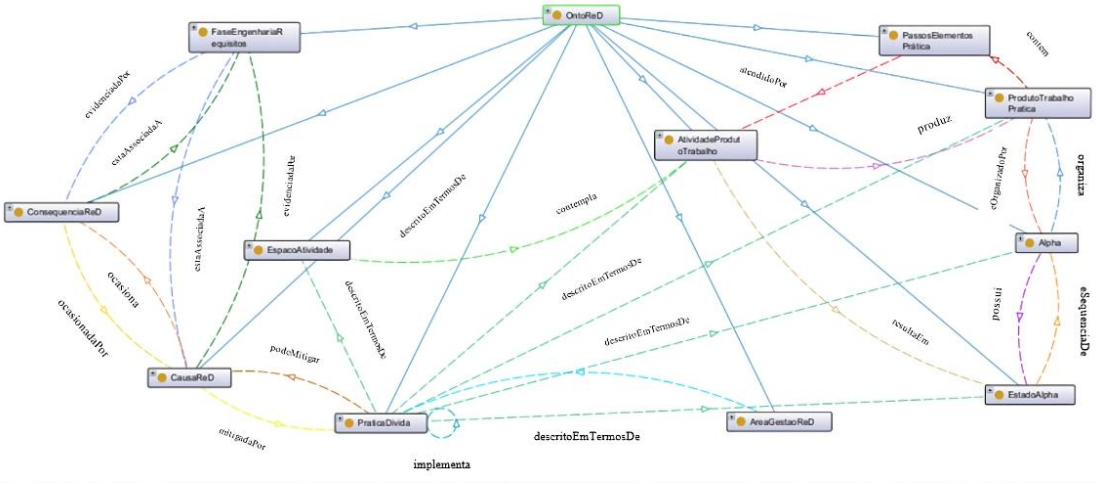
Fonte: (Horridge; Brandt, 2011)

FIGURA 31 - *Objects Properties*



Fonte: Capturada do Protégé – OntoReD (2023)

FIGURA 32 - Classes da OntoReD implementadas no Protegé e exemplos da relação entre elas por meio das *Objects Properties*



Fonte: Capturada do Protégé – OntoReD (2023)

QUADRO 4 - *Objects Properties Domain e Range*

Domain	Propriedade	Range
EstadoAlpha	avancadoPor	AtividadeProdutoTrabalho
AtividadeProdutoTrabalho	resultaEm	EstadoAlpha
FaseEngenhariaRequisitos	eEvidenciadaPor	ConsequenciaReD
FaseEngenhariaRequisitos	eEvidenciadaPor	CausaReD
ConsequenciaReD	estaAssociadaA	FaseEngenhariaRequisitos
CausaReD	estaAssociadaA	FaseEngenhariaRequisitos
ProdutoTrabalhoPratica	eOrganizadoPor	Alpha
Alpha	organiza	ProdutoTrabalhoPratica
ProdutoTrabalhoPratica	eProduzidoPor	AtividadeProdutoTrabalho
AtividadeProdutoTrabalho	produz	ProdutoTrabalhoPratica
AtividadeProdutoTrabalho	estaContempladaEm	EspacoAtividade
EspacoAtividade	contempla	AtividadeProdutoTrabalho
PraticaDivida	implementa	PraticaDivida
PraticaDivida	implementadaPor	PraticaDivida
CausaReD	mitigadoPor	PraticaDivida
PraticaDivida	podeMitigar	CausaReD
CausaReD	ocasiona	ConsequenciaReD
ConsequenciaReD	ocasionadaPor	CausaReD
Alpha	possui	EstadoAlpha
EstadoAlpha	eSequenciaDe	Alpha
PraticaDivida	descritoEmTermosDe	Alpha
PraticaDivida	descritoEmTermosDe	EstadoAlpha
PraticaDivida	descritoEmTermosDe	AtividadeProdutoTrabalho
PraticaDivida	descritoEmTermosDe	EspacoAtividade
PraticaDivida	descritoEmTermosDe	ProdutoTrabalhoPratica
AreaGestaoReD	eConstituidaDe	PraticaDivida
PraticaDivida	eParteDe	AreaGestaoReD
Alpha	progredidoPor	AtividadeProdutoTrabalho
PassosElementosPratica	atendidoPor	AtividadeProdutoTrabalho
ProdutoTrabalhoPratica	contem	PassosElementosPratica
PassosElementosPratica (Níveis de detalhes)	estaContidoEm	ProdutoTrabalhoPratica

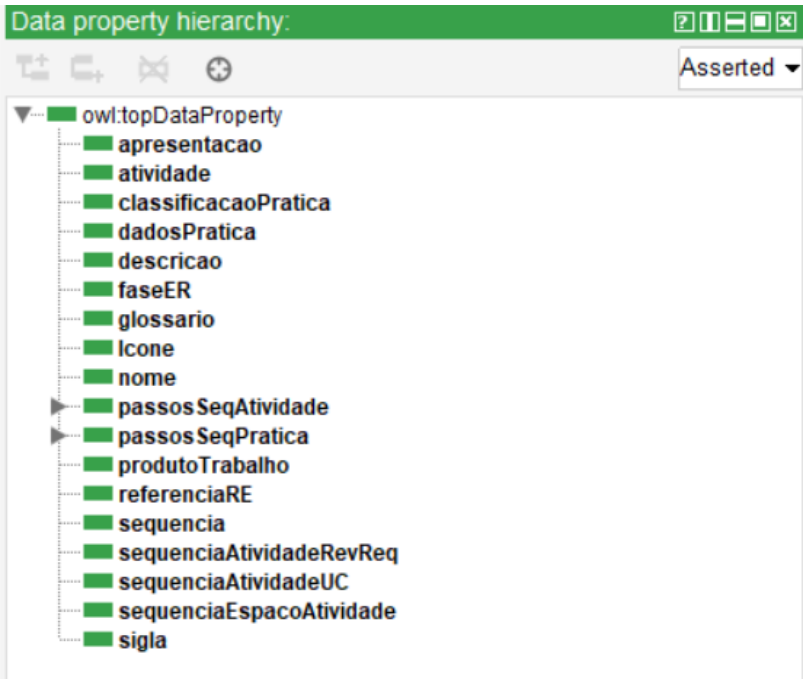
AtividadeProdutoTrabalho	acessa	PraticaDivida
PraticaDivida	acessadoPor	AtividadeProdutoTrabalho

Fonte: Elaborado pelo autor (2023)

2.6.3 Definição dos atributos de instâncias da OntoReD – Data Properties

A definição dos atributos das instâncias consiste em representar os dados necessários para cada instância de um conceito (classes), os quais podem ser visualizados na figura 33.

FIGURA 33 - *Data property* das instâncias



Fonte: Capturada do Protégé – OntoReD (2023)

2.6.4 Axiomas

Os axiomas especificam as regras assumidas como verdadeiras em um determinado domínio, nas quais associa-se os relacionamentos entre os conceitos (classes) e consequentemente, suas instâncias, além de fornecer características descritivas e lógicas para os conceitos (Fernández-Pérez; Goméz- López; Corcho, 2004).

Para definir os axiomas relevantes na Ontologia, a metodologia prevê especificar as seguintes informações: nome, descrição, expressão lógica que descreve formalmente o axioma,

conceitos, atributos e relações voltadas às variáveis presentes no axioma, as quais estão sintetizadas no exemplo apresentado no Quadro 5 (Fernández-Pérez; Goméz- López; Corcho, 2004).

QUADRO 5 - Axioma voltado às práticas que podem mitigar as causas das dívidas de requisitos

Nome da regra	Práticas que podem mitigar as causas das dívidas de requisitos considerando a área de gestão da dívida
Descrição	Esta regra é responsável por apresentar as práticas que podem mitigar as causas das dívidas de requisitos, bem como suas consequências, categorizadas por áreas da gestão da dívida
Expressão	if [CausaReD] (?Y) [ocasiona] [ConsequenciaReD] (?X) and [CausaReD] (?Y) [mitigadoPor] [PraticaDivida] (?Z) and [AreaGestaoReD] (?W) [eConstituidaDe] [PraticaDivida] (?Z) then [nome] (?W, “nome da instância”, ?Z, “nome da instância”)
Conceitos	CausaReD, ConsequenciaReD, PraticaDivida, AreaGestaoDividaReD
Atributos	Nome da Área da Gestão da Dívida, Nome da Prática da dívida de requisitos
Relação	ocasiona, mitigadoPor, eConstituidaDe
Variáveis	?W ?Z

Fonte: Elaborado pelo autor (2023)

2.6.5 Definição das instâncias das Classes da OntoReD

As instâncias dos conceitos da OntoReD formam a sua base de conhecimento e deve manter um nome, a classe a qual faz parte e os valores de atributos. O Quadro 6 apresenta um exemplo de uma instância e o conceito correspondente.

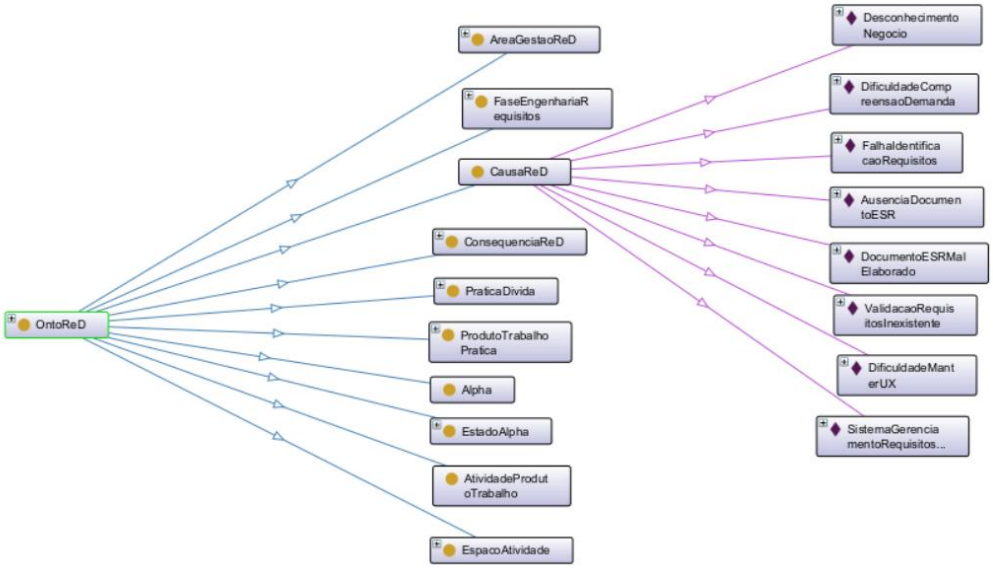
QUADRO 6 - Exemplos de Instâncias da OntoReD

Nome da Instância	Nome da Classe
ValidacaoRequisitosInexistente	CausaReD
ImplementacaoIncompativelComEspecificacao	ConsequenciaReD
RevisaoRequisitos	PraticaDivida
Prevencao	AreaGestãoReD
ValidaçãodeRequisitos	FaseEngenhariaRequisitos

Fonte – Elaborado pelo autor (2023)

Visando demonstrar um exemplo de instâncias de um determinado conceito, apresenta-se na Figura 34 a classe “CausaReD” e suas instâncias correspondentes.

FIGURA 34 - Exemplos de Instâncias da classe “CausaReD” da OntoReD



Fonte: Capturada do Protégé (2023)

Após as atividades orientadas ao desenvolvimento previstas na *Methontology*, são propostas atividades de suporte, como a atividade de aquisição de conhecimento, integração (não aplicada), avaliação, documentação e gerenciamento de configuração (Iqbal *et al.* 2013; Fernández-Pérez; Goméz- López; Corcho, 2004).

2.7 Avaliação da OntoReD

A avaliação proposta na *Methontology*, sinaliza um documento que registre a avaliação, na qual o pesquisador deve descrever como a Ontologia foi avaliada, as técnicas utilizadas, o tipo de erros encontrados e as fontes de conhecimento utilizadas na avaliação e verificar a sua consistência (Fernández-López; Gomés-Pérez; Juristo, 1997).

Na OntoReD foi realizada a sua avaliação técnica pela ferramenta, e seguindo essa característica, a sua consistência foi verificada na opção *Debugger* no Protégé, sendo que todas as inconsistências apontadas foram resolvidas. Para auxiliar na verificação da consistência (*consistency checking*), foi utilizado o mecanismo de inferência (*Reasoner*) Hermit disponível

no Protégé para essa finalidade (Horridge, 2008) e que permite a especificação e execução de consultas na ontologia. A avaliação da OntoReD também foi realizada por meio do conhecimento de especialistas.

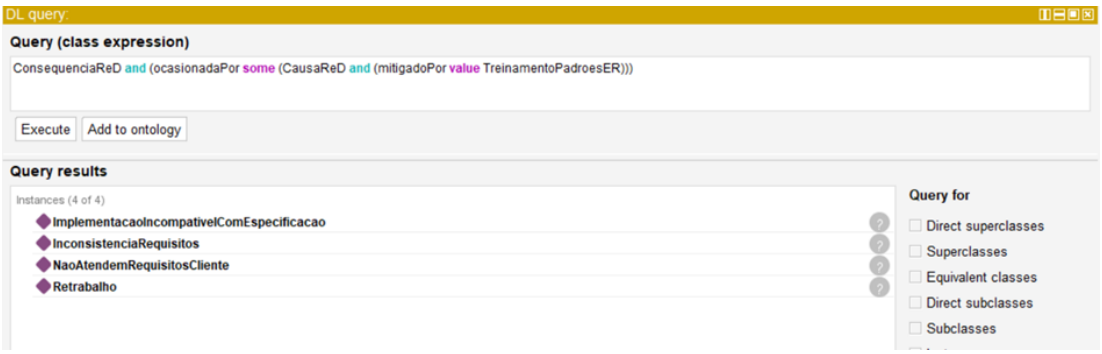
Para demonstrar visualmente a consistência e a coerência da OntoReD, apresenta-se exemplo de consulta como prova de conceito. A consulta, apresentada no Quadro 7, foi definida inicialmente em linguagem natural e, após, convertida em uma DL Query e executada no Protégé por meio do mecanismo de inferência. O resultado da DL Query pode ser visualizado na Figura 35.

QUADRO 7 - Exemplo de consulta da OntoReD em DL Query

Consulta 1	***Quais são as consequências da dívida de requisitos que podem ser mitigadas pela prática "Treinamento de padrões de elicitação de requisitos"?***
DL Query 1	*ConsequenciaReD and (ocasionadaPor some (CausaReD and (mitigadoPor value TreinamentoPadroesER)*
Resultado 1	*Figura 35*

Fonte: Elaborado pelo autor (2023)

FIGURA 35 - Resultado Consulta 1: Consequências da dívida de requisitos mitigadas pela prática "Treinamento de padrões de elicitação de requisitos"



Fonte: Capturada do Protégé (2023)

Verificada a consistência da OntoReD, a formalização e, posteriormente, a implementação permitiu obter as respostas para as questões definidas no Quadro 14, da seção 5.4.1, conforme demonstra o Quadro 8. As Figuras 36, 37, 38, 39, 40, 41 e 42 demonstram as consultas na ferramenta Protegé referente às questões apresentadas no Quadro 1, Seção 2.3.

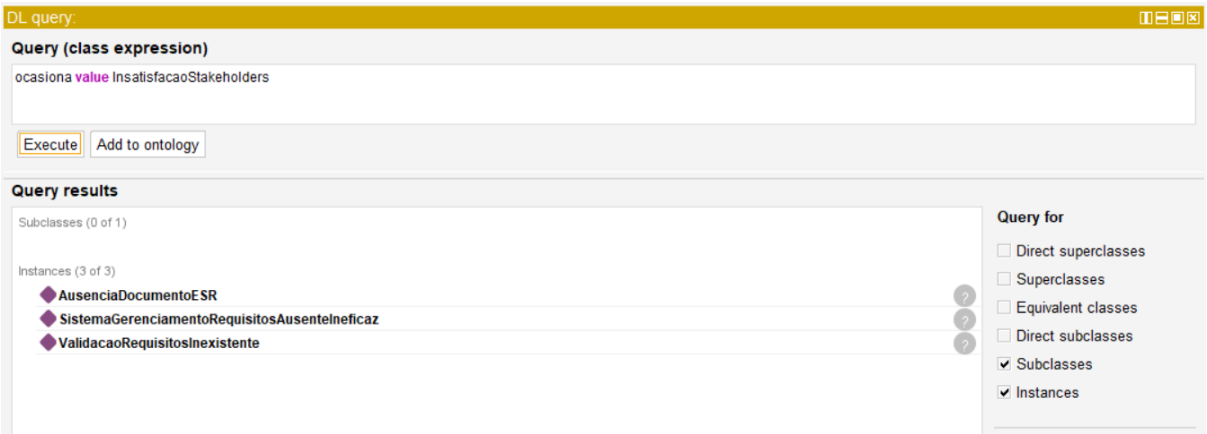
QUADRO 8 - Respostas às questões previstas na especificação da OntoReD, conforme Quadro 1, Seção 2.3

Questões respondidas pela OntoReD	**Consulta na ontologia**	**Respostas às consultas**	**Referência**
Q1. Quais causas da dívida ocasiona(m) determinada Consequência?	ocasiona value InsatisfacaoStakeholders n	AusenciaDocumentoESR; SistemaGerenciamentoRequisitosAusenteIneficaz; ValidacaoRequisitosInexistente	Figura 36
Q2. Quais consequências são causadas por determinada causa?	ocasionadaPor value SistemaGerenciamentoRequisitosAusenteIneficaz ——————— ConsequenciaReD and (ocasionadaPor some (CausaReD and (mitigadoPor value DesenvolverPrototipoDocumentoEREQ)))	ImplementacaoIncompativelComEspecificacao; InconsistenciaRequisitos; InsatisfacaoStakeholders; NaoAtendemRequisitosCliente; Retrabalho ——————— DocumentoESRInadequada; InsatisfacaoStakeholders; NaoAtendemRequisitosCliente;	Figuras 37 e 38
Q3. Quais são as práticas que podem mitigar determinada(s) causa(s) da dívida de requisitos?	podeMitigar value AusenciaDocumentoESR	DesenvolverPrototipoDocumentoEREQ; ManterDescreverClarezaDREQ; PrototipoValidacaoREQ	Figura 39
Q4. Quais são as áreas que permitem a gestão da dívida de requisitos? e Q5. Quais práticas estão associadas a determinada área da gestão da dívida?	implementa some ((nome value "Ações preventivas") and (eParteDe value Identificacao))	CompromissoCliente; ComunicacaoPartesInteressadasStakeholders; ControlarNegociarREQ; DeclaracaoEscopoDefinido; RastreabilidadeREQ; RequisitosBemDefinidos	Figuras 40 e 41

Q6. Qual área da gestão da dívida está associada a determinada prática que pode mitigar determinada causa?	AreaGestaoReD and (eConstituidaDe value MelhorarProcessoDesenvolvimento)	Prevencao	Figura 42

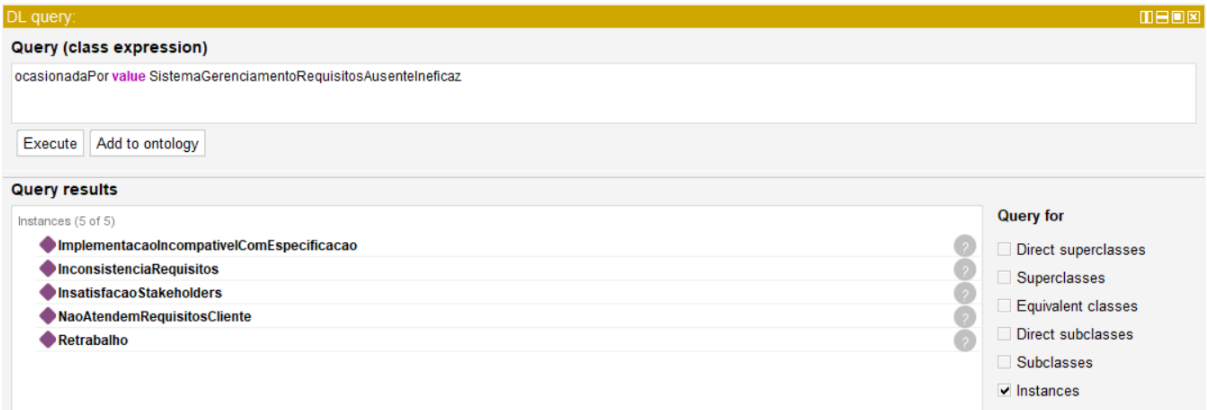
Fonte: Consultas obtidas do Protégé realizadas pelo autor (2023)

FIGURA 36 - Resultado Q1. Quais causas da dívida ocasiona(m) determinada Consequência?



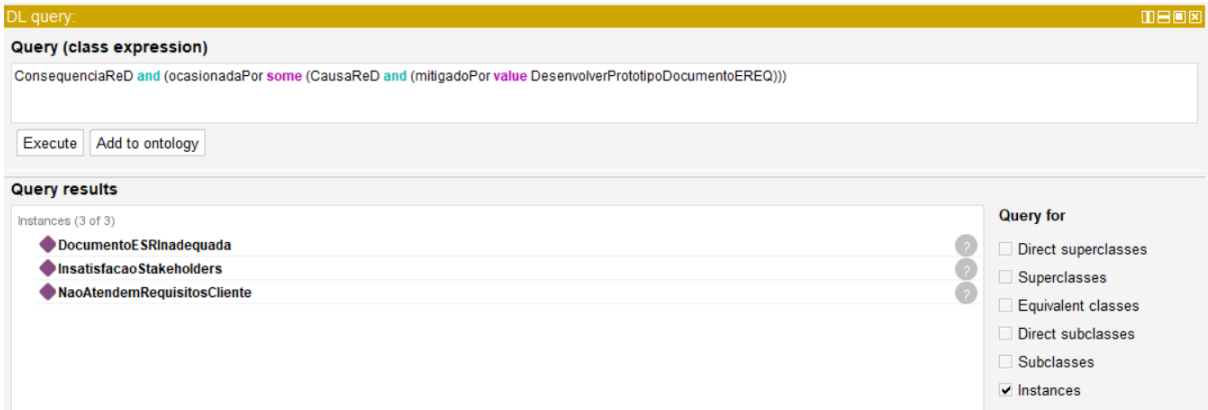
Fonte: Capturada do Protégé (2024)

FIGURA 37 - Resultado Q2. Quais consequências são causadas por determinada causa?



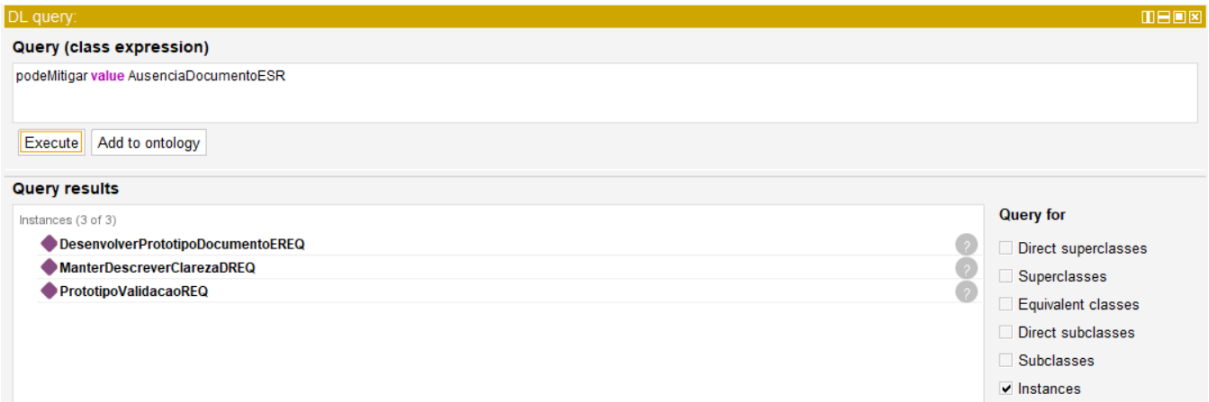
Fonte: Capturada do Protégé (2024)

FIGURA 38 - Resultado Q2. Quais consequências são causadas por determinada causa?



Fonte: Capturada do Protégé (2024)

FIGURA 39 - Resultado Q3. Quais são as práticas que podem mitigar determinada(s) causa(s) da dívida de requisitos?

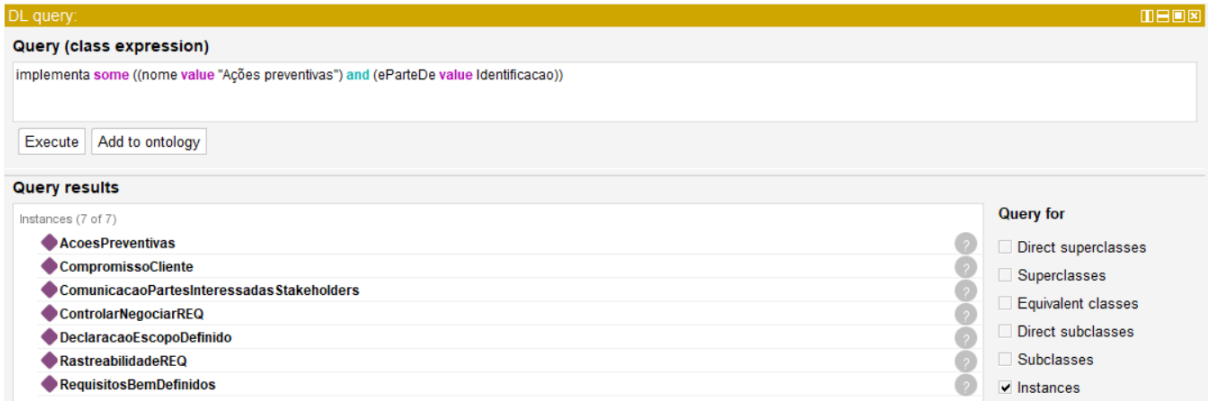


Fonte: Capturada do Protégé (2024)

FIGURA 40 - Resultado Q4. Quais são as práticas que podem mitigar determinada(s) causa(s) da dívida de requisitos?

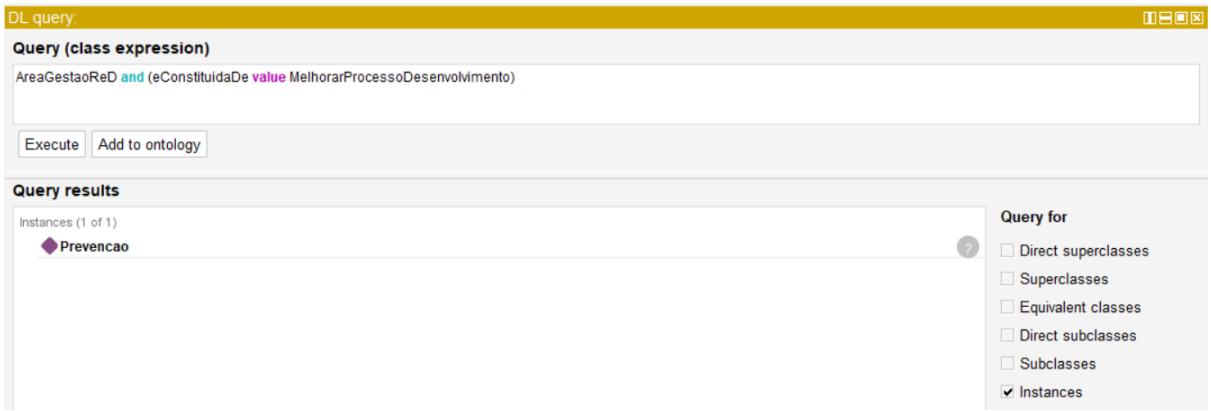
Fonte: Capturada do Protégé (2024)

FIGURA 41 - Resultado Q5. Quais práticas estão associadas a determinada área da gestão da dívida?



Fonte: Capturada do Protégé (2024)

FIGURA 42 - Resultado Q6. Qual área da gestão da dívida está associada a determinada prática que pode mitigar determinada causa?



Fonte: Capturada do Protégé (2024)

2.8 Documentação da OntoReD

A documentação da OntoReD é pautada, muitas vezes, no código da Ontologia, pois não há diretrizes que formalizam essa documentação, incluindo publicações realizadas em anais e conferências para oficializar essa documentação. No entanto, na *Methontology* propõe-se a atividade de documentação durante todo o processo de desenvolvimento da ontologia (Fernández-López; Gomés-Pérez; Juristo, 1997).

A documentação da OntoReD é pautada no registro de todas as atividades do processo de desenvolvimento da Ontologia, bem como a própria ontologia e está disponível em https://github.com/vivi-duarte/PracticesforReD/tree/main/OntoReD.

2.9 Glossário da OntoReD

O Quadro 9 apresenta o Glossário da OntoReD.

QUADRO 9 - Exemplos de Instâncias da OntoReD

Nome	Termo/Sigla	Descrição	Tipo
Ontologia voltada à dívida de requisitos	OntoReD	Ontologia que representa o conhecimento voltado à dívida de requisitos	Conceito
Dívida de Requisitos	ReD	"refere-se à distância entre a especificação de requisitos desejada e a real implementação do sistema, sob suposições e restrições de domínio" (Ernst, 2012).	Conceito
Fase da Engenharia de Requisitos	FaseEngenhariaRequisitos	Fases que compõem a Engenharia de Requisitos.	Conceito
Causa da dívida de requisitos	CausaReD	Fator eficaz que dá existência ao que não existia, ou seja, a origem das consequências das dívidas de requisitos.	Conceito
Consequência da dívida de requisitos	ConsequenciaReD	Problemas relacionados ao processo da Engenharia de Requisitos resultantes das causas, e que se manifestam no processo de desenvolvimento.	Conceito
Área da gestão da dívida de requisitos	AreaGestaoReD	A área da gestão da dívida consiste em fundamentar as atividades que apoiam especialmente o monitoramento, priorização, prevenção e pagamento da dívida. Áreas se estendem às dívidas de Requisitos (Albuquerque, 2022).	Conceito
Prática para mitigar a dívida	PraticaDívida	"Uma prática é uma abordagem repetível para fazer algo com um objetivo específico em mente. Uma prática descreve como lidar com um aspecto específico de um empreendimento de engenharia de software, incluindo as descrições de todos os elementos relevantes necessários para expressar a orientação de trabalho desejado para atingir o objetivo da prática. Uma prática pode ser definida como uma composição de outras práticas" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	Conceito
Produto de trabalho da prática	ProdutoTrabalhoPratica	"Um produto de trabalho é um artefato de valor e relevância para um empreendimento de engenharia de software. Um produto de trabalho pode ser um documento ou um pedaço de software, ou outras entidades criadas." (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	Conceito
Atividade do produto de trabalho	AtividadeProdutoTrabalho	"Uma atividade define uma ou mais abordagens para executar algum trabalho a ser executado e pode recomendar ações sobre alfas e/ou produtos de trabalho para executar esse trabalho." (Jacobson, 2018). Disponível em:	Conceito

		https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	
Espaço atividade	EspacoAtividade	"Um espaço reservado para algo a ser feito no empreendimento de engenharia de software." (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	Conceito
Passos dos elementos da prática	PassosElementosPrática	Refere-se aos elementos que representam os níveis de detalhes da prática.	Conceito
Alpha	Alpha	"Um alfa representa e mantém o estado de algum elemento, aspecto ou abstração em um empreendimento que tem um estado discernível e o conhecimento de cujo estado é necessário para entender o estado de progresso e/ou saúde do empreendimento" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	Conceito
Estado alpha	EstadoAlpha	"Um estado alfa é uma especificação do estado de progresso de um alfa. O estado de um alfa é determinado pela avaliação de itens da lista de verificação" (Jacobson, 2018). Disponível em: https://practicelibrary.ivarjacobson.com/content/essence-language-key Acesso em Maio de 2023.	Conceito
Nome	nome	Denominação de todos os elementos do domínio.	Atributo de instância
Descrição	descricao	Apresenta a descrição dos elementos da Ontologia	Atributo de instância
Classificação da Prática	classificacaoPratica	Define a classificação da prática da dívida: geral, comum genérica –(quando sugerida para a todas as causas), e específica (voltada à algumas causas).	Atributo de instância
Apresentação da prática	apresentacaoPratica	Apresenta uma visão geral sobre a Prática.	Atributo de instância
Produto de trabalho	produtoTrabalho	Artefato que compõe uma prática	Atributo de instância
Atividade	atividade	Ação que atende a elaboração ou parte de um artefato	Atributo de instância
Está associada a	estaAssociadaA	Causa da ReD está associada a uma fase da engenharia de requisitos; Consequência da ReD está associada a uma fase da engenharia de requisitos.	Relação
É evidenciada por	eEvidenciadaPor	Consiste em evidenciar as causas e consequências associadas às fases da Engenharia de Requisitos.	Relação
Ocasiona	ocasiona	Motiva o acontecimento de algo, por exemplo: Uma causa ocasiona uma ou várias consequências.	Relação
Ocasionada por	ocasionadaPor	Representa a relação consequência -> causa.	Relação
Pode mitigar	podeMitigar	Representa que algo pode ser minimizado, por exemplo, causa que pode ser minimizada com alguma ação.	Relação
Mitigado por	mitigadoPor	Consiste em causas que podem ser mitigadas por determinadas práticas.	Relação

É constituída de	eConstituidaDe	Condiciona a colaboração de vários elementos para alcançar determinado resultado, por exemplo:	Relação
É Implementada Por	implementadaPor	Garante a realização de algo.	Relação
implementa	implementa	Prática da dívida implementa uma outra prática da dívida	Relação
Acessa	acessa	Refere-se às atividades de produto de trabalho que podem acessar determinadas práticas da dívida.	Relação
Acessado por	acessadoPor	Refere-se ao conjunto de práticas da dívida que podem ser acessadas por atividades de produto de trabalho.	Relação
Atendido por	atendidoPor	Consiste no atendimento dos passos dos elementos da prática pelas atividades do produto de trabalho.	Relação
Avançado por	avancadoPor	Evidências do estado alpha que pode ser avançado por atividades do produto de trabalho.	Relação
Contém	contem	Refere-se as evidências do produto de trabalho que contém os passos dos elementos da prática.	Relação
Contempla	contempla	Evidências do espaço atividade que contempla atividades do produto de trabalho.	Relação
Descrito em termos de	descritoEmTermosDe	Refere-se às evidências da Prática da ReD que são descrtias em termos do: produto de trabalho da prática; espaço atividade; atividade do produto de trabalho; alpha; e estado alpha.	Relação
É organizado por	eOrganizadoPor	Produto de trabalho da prática é organizado por alpha.	Relação
É parte de	eParteDe	Prática da ReD é parte da área da gestão da dívida.	Relação
É produzido por	eProduzidoPor	Produto de trabalho da prática é produzido por atividade do produto de trabalho.	Relação
É progredido por	eProgredidoPor	Alpha é progredido por atividade do produto de trabalho.	Relação
É Sequência de	eSequenciaDe	Estado alpha é sequência do elemento alpha.	Relação
Está contemplada em	estaContempladaEm	Atividade do produto de trabalho está contemplada em espaço atividade.	Relação
Está contido em	estaContidoEm	Passos dos elementos da prática está contido em Produti de trabalho da prática.	Relação
Organiza	organiza	Elemento alpha que organiza o produto de trabalho da prática.	Relação
Possui	possui	Elemento alpha possui estados alpha.	Relação
Produz	produz	Atividade do produto de trabalho produz produto de trabalho da prática.	Relação
Resulta em	resultaEm	Atividade do produto de trabalho resulta em estado alpha	Relação

Fonte: Elaborado pelo autor (2023)

REFERÊNCIAS

ALBUQUERQUE, Danyllo; GUIMARAES, Everton; TONIN, Graziela; PERKUSICH, Mirko; ALMEIDA, Hyggo; PERKUSICH, Angelo. **Comprehending the Use of Intelligent Techniques to Support Technical Debt Management.** IEEE/ACM International Conference on Technical Debt (TechDebt). 2022. DOI: 10.1145/3524843.3528097. Disponível em: https://ieeexplore.ieee.org/document/9804504. Acesso em: Jan. 2023.

ALSANAD, Abeer Abdulaziz; CHIKH, Azeddine; MIRZA, Abdulrahman. **A Domain Ontology for Software Requirements Change Management in Global Software Development Environment**. DOI: DOI: 10.1109/ACCESS.2019.2909839. 2019. Disponível em: https://ieeexplore.ieee.org/document/8684236. Acesso em: Fev. 2023.

BONFIM, Viviane Duarte; BENITTI, Fabiane Barreto Vavassori. **OntoReD: Requirements Debt Ontology**. Requirements Engineering, 2026. No prelo.

DUARTE, Luiz. **Scrum e Métodos Ágeis - Um Guia Prático**. (2021). 2. Ed. ISBN: 978-65-900538-5-5.

FERNÁNDEZ-LÓPEZ, Mariano; GÓMEZ-PÉREZ, Asunción; JURISTO, Natalia. **METHONTOLOGY: from ontological art towards ontological engineering**. 1997. Disponível em: : https://www.researchgate.net/publication/50236211. Acesso em: Nov. 2022.

FERNÁNDEZ-LÓPEZ, Mariano; GÓMEZ-PÉREZ, Asunción; CORCHO, Oscar. **Ontological Engineering with examples from the areas of Knowledge Management, e-Commerce and the Semantic Web**. © Springer-Verlag London Limited 2004. ISBN 1-85233-551-3.

FOWLER, Martin Fowler, Martin. **UML essencial: um breve guia para a UML essencial** [recurso eletrônico] : um breve guia para a linguagem-padrão de modelagem de objetos / Martin Fowler ; linguagem-padrão de modelagem de objetos / Martin Fowler ; tradução João tradução João Tortello. – 3. ed. – Dados eletrônicos. – Porto ortello. – 3. ed. – Dados eletrônicos. – Porto Alegre : Bookman, 2007. Alegre : Bookman, 2007. Esta edição abrange a versão UML Esta edição abrange a versão UML 2.0 OMG 2.0 OMG. ISBN 978-85-60031-38-2.

GÓMEZ-PÉREZ, Asunción; FERNÁNDEZ-LÓPEZ, Mariano; CORCHO, Oscar. **Ontological Engineering: with examples from the areas of Knowledge Management**, e-Commerce and the Semantic Web. London: Springer London, 2004.

HORRIDGE, Matthew. **Um guia prático para a construção de ontologias OWL, plugin Protégé-OWL 3.4**. Tradução de SOARES, D.R.; ALMEIDA, M.B. 2008.

Disponível em: http://mba.eci.ufmg.br/onto_owl.

HORRIDGE, Matthew; BRANDT, Sebastian**. A practical guide to building OWL ontologies using the Protégé-OWL plugin and CO-ODE tools (v.1.3).** University of Manchester, 2011.

INTERNATIONAL STANDARD. ISO/IEC/ IEEE 29148 Reference number **ISO/IEC/IEEE 29148:2018** - Systems and software engineering — Life cycle processes. ISO/IEC/IEEE, 29148, 2018.

JACOBSON, Ivar. **Practice Library**. 2018. Disponível em https://practicelibrary.ivarjacobson.com/. Acesso: 26 jun. de 2021.

JACOBSON, Ivar; STIMSON, Roly. **The Essence of Software Engineering: Escaping Method Prison – On the Road to Real Software Engineering.** p. 37-58, 2018. Disponível em: https://link.springer.com/chapter/10.1007/978-3-319-73897-0_3. Acesso em: Abr. 2021.

IEEE. ISO/IEC/IEEE 29148. International Standard. **Systems and software engineering - Life cycle processes - Requirements engineering** (2018).

IQBAL, Rizwan; MURAD, Masrah Azrifah Azmi; MUSTAPHA, Ainda; SHAREF, Nurfadhlina Mohd. **An Analysis of Ontology Engineering Methodologies: A Literature Review.** In: Research Journal of Applied Sciences, Engineering and Technology. 2013. DOI: 10.19026/rjaset.6.3684.

MIZOGUCHI, Riichiro. **Ontological engineering: foundation of the next generation knowledge processing**. In: SPRINGER. ASIA-PACIFIC Conference on Web Intelligence. [S.l.: s.n.], 2001. p. 44–57.

NOY, Natalya F. et al. **Ontology development 101**: A guide to creating your first ontology. 2001.

OLIVEIRA Jr, Edson; GERALDI, Ricardo Theis. (2017). **Defect Types and Software Inspection Techniques: A Systematic Mapping Study**. Journal of Computer Science, 13(10), 470-495. https://doi.org/10.3844/jcssp.2017.470.495.

OMG, OBJECT MANAGEMENT GROUP. **Essence – Kernel and Language for Software Engineering Methods**. Versão 1.2. 2018. Disponível em: https://www.omg.org/spec/Essence/20150601/Essence.xmi. Acesso em: Nov. 2022.

PEREIRA-VALE, Anelis; FERNANDÉZ, Eduardo B. **An Ontology for Security Patterns**. In: 38th International Conference of the Chilean Computer Science Society. 2019. DOI: 10.1109/SCCC49216.2019.8966393.

POHL, Klaus; Rupp, Chris. **Fundamentos da Engenharia de Requisitos** (2015). International Requirements Engineering Board.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. 9th ed. Porto Alegre: AMGH, 2021. E-book. p.102. ISBN 9786558040118. Disponível em:

https://integrada.minhabiblioteca.com.br/reader/books/9786558040118/. Acesso em: 10 out. 2024.

REINEHR, Sheila. **Engenharia de Requisitos** (2020). ISBN 978-65-5690-067-4. Porto Alegre: SAGAH.

SOMMERVILLE, Ian. **Software Engineering**. (2018). 10. Ed. ISBN: 978-85-430-2497-4.

VAZQUEZ, Carlos Eduardo; SIMÕES, Guilherme Siqueira. **Engenharia de Requisitos: software orientado ao negócio**. Brasport, 2016.

WASHIZAKI, Hironori**. Guide to the Software Engineering Body of Knowledge**.

IEEE Computer Society, 2024.

WIEGERS, Karl; BEATTY, Joy. **Software Requirements**. Third Edition ISBN: 978-0-7356-7966-5. Redmond – Washington. ISBN: 978-0-7356-7966-5, 2013.